Chapter 10: MLR Diagnostics for Causal Inference

<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to evaluate the five core regression assumptions (normality, multicollinearity, autocorrelation, linearity, homoscedasticity) using appropriate diagnostic tests and plots <{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to detect and address multicollinearity using correlation heatmaps and Variance Inflation Factor (VIF) analysis <{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to apply label transformations (log, Box-Cox, Yeo-Johnson) to correct residual normality violations <{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to diagnose linearity violations through residual-versus-fitted plots and address them using polynomial features or transformations <{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to explain why regression diagnostics are essential for valid causal inference but less critical for purely predictive modeling

10.1Introduction

In the previous chapter, you learned how to build multiple linear regression (MLR) models for *causal (explanatory) analysis*. You focused on estimating coefficients, interpreting feature effects, and understanding model fit using statistics such as R^2, *Adjusted* R^2, *t*-values, and *p*-values. Those tools allowed you to ask questions like: *Which features matter?* and *How strong is their relationship with the outcome?*

This chapter shifts the focus from *estimating* a regression model to *evaluating and refining* one for the purpose of reliable explanation. The central question is no longer just "What does the model say?" but "*When should we trust what the model says as evidence about relationships in the world?*" Answering that question requires understanding the assumptions that underlie linear regression and learning how to diagnose when those assumptions are violated.

Regression assumptions exist to support valid *statistical inference*. They ensure that coefficient estimates are stable, standard errors are meaningful, and hypothesis tests behave as expected. When these assumptions hold reasonably well, regression results can be interpreted as credible evidence about relationships among variables. When they do not, coefficient estimates may still exist, but their interpretation becomes unreliable.

In contrast, many modern analytics projects emphasize *prediction* rather than explanation. In predictive settings, the goal is not to understand why an outcome occurs, but to forecast future values as accurately as possible. In those contexts, some regression assumptions can be relaxed or ignored entirely, as long as predictive performance is validated on new data.

This chapter adopts a clear organizing principle: **regression diagnostics matter most when the goal is causal or explanatory modeling**. You will learn which assumptions are essential for trustworthy inference, which violations are especially dangerous for interpretation, and why some fixes that improve prediction may actually undermine causal clarity.

A central theme of this chapter is *diagnostic-driven feature design*. Instead of adding, removing, or transforming variables to maximize fit alone, you will use diagnostics—such as residual plots, normality tests, and variance inflation factors (VIF)—to make principled adjustments that improve interpretability, stability, and inferential validity.

Throughout the chapter, you will still see references to metrics like R^2 and *Adjusted* R^2, but always in service of explanation rather than prediction. In particular, you will learn why a lower R^2 can sometimes reflect a better causal model when it aligns more closely with the assumptions required for inference.

By the end of this chapter, you will be able to diagnose regression problems, refine models systematically, and judge when regression results can be responsibly interpreted as explanatory evidence. In the next chapter, you will revisit many of these same tools from a different perspective—optimizing models for prediction, where the trade-offs and priorities change.

10.2Regression Diagnostics

Why Assumptions Exist

In the previous chapter, you learned how to build and interpret multiple linear regression (MLR) models for *explanatory (causal)* purposes. This chapter focuses on the next critical step:

regression diagnostics. Diagnostics help you evaluate whether the mathematical conditions that justify coefficient interpretation and statistical inference are reasonably satisfied.

Regression assumptions exist for both *mathematical* and *practical* reasons. Mathematically, they ensure that the estimators produced by ordinary least squares (OLS) have desirable properties such as unbiasedness, efficiency, and valid standard errors. Practically, they determine whether quantities like coefficients, p-values, and confidence intervals can be interpreted as reliable evidence about relationships in the data.

When assumptions fail, the regression model does not suddenly become useless. Instead, *specific components of the output become unreliable*. Diagnostics help you identify *what is breaking*, *why it is breaking*, and *which modeling adjustments are appropriate* when the goal is explanation rather than prediction.

The table below summarizes the core regression assumptions examined in this chapter, what breaks when each assumption is violated, why that failure matters for causal interpretation, and a concrete example to anchor each concept.

Table 10.1
Why Regression Assumptions Matter

Assumption	What Breaks When Violated	Why It Matters for Inference	Example
Normality	p-values and confidence intervals	Statistical inference becomes unreliable	Residuals are heavily right-skewed, inflating t-tests
Multicollinearity	Coefficient stability	Feature effects cannot be disentangled	Two predictors move together and swap signs across models
Autocorrelation	Standard errors	False confidence in results	Daily sales residuals are correlated across time
Linearity	Model specification	Systematic bias in estimates	Price increases exponentially but is modeled linearly
Homoscedasticity	Efficiency of estimates	Uncertainty is misestimated	Prediction errors grow as values increase

Regression diagnostics should not be treated as pass–fail tests. Real-world data rarely satisfies all assumptions perfectly, and attempting to enforce strict compliance often leads to unnecessary complexity or distorted interpretation.

Instead, diagnostics should be understood as *signals* that guide judgment-driven modeling decisions:

- Diagnostics indicate *where* causal interpretation may be fragile.
- Diagnostics suggest *which features* may require transformation, re-specification, or removal.
- Diagnostics help distinguish *inference problems* from issues that primarily affect prediction.
- Diagnostics support *analytical judgment*, not automatic correction.

Throughout this chapter, you will learn how to evaluate each assumption, interpret its diagnostic signals, and decide when corrective action is necessary for explanatory modeling. You will also see that *prediction-oriented models can often tolerate assumption violations that causal (inference-oriented) models cannot*, a distinction that becomes the central focus of the next chapter.

10.3Normality

Normality is one of the most commonly discussed—and most commonly misunderstood—assumptions in regression modeling. It is often incorrectly described as a requirement that the data itself be normally distributed.

In reality, normality plays a much narrower role. It primarily affects *statistical inference*, not the model's ability to generate point predictions. When normality assumptions are reasonably satisfied, we can trust p-values, confidence intervals, and hypothesis tests on coefficients. When they are violated, predictions may still be useful, but inference becomes less reliable—making this assumption especially important for *causal (explanatory)* modeling.

In this section, we treat normality diagnostics as *signals rather than pass/fail tests*. Our goal is not to achieve perfect normality, but to understand what deviations tell us about model behavior and how to respond in ways that protect interpretability and inference.

Univariate Normality (Label Distribution)

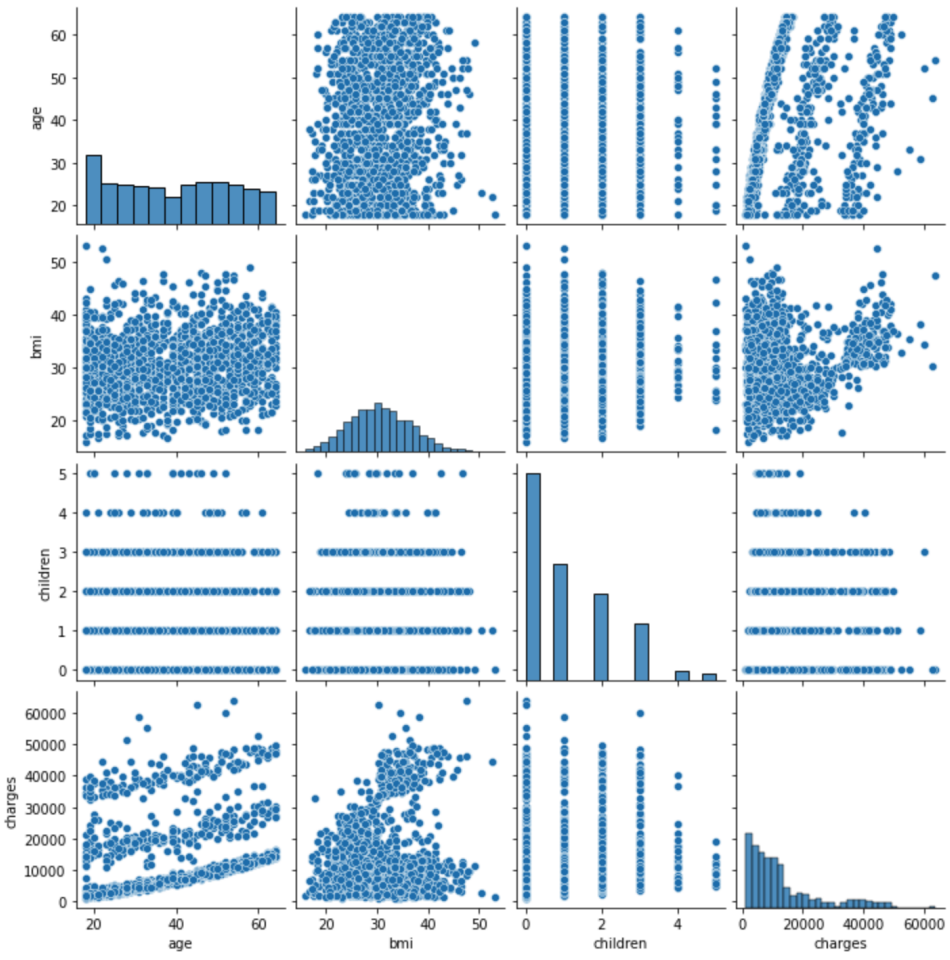
The *OLS().summary()* output reports skewness and kurtosis for the dependent variable. In this dataset, the label *charges* exhibits substantial right skew and elevated kurtosis, indicating a heavy-tailed distribution.

Strictly speaking, regression does not require the label itself to be normally distributed. However, extreme skewness in the label often carries downstream consequences that matter for

explanatory modeling: it can contribute to non-normal residuals, unstable variance, and coefficient tests that are less trustworthy.

Because of this indirect effect, it is often useful to inspect the label distribution visually using histograms and pair plots rather than relying only on numeric skewness statistics.

```
import pandas as pd
import seaborn as sns

df = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/data/insurance.csv')

sns.pairplot(df[['age', 'bmi', 'children', 'charges']]);
```



The diagonal of the pair plot shows the marginal distributions of each numeric variable. Binary features and discrete count variables are not expected to follow a normal distribution and are not a concern. In this dataset, *charges* is clearly right-skewed, while features such as *bmi* appear approximately normal.

When a label is strongly skewed, mathematical transformations can improve symmetry and stabilize variance. The goal is not to make the data “perfectly normal,” but to reduce extreme skew that can distort residual behavior and, in turn, weaken inference about coefficients.

A simple starting point is to try a few intuitive transformations such as square root, cube root, and natural log. These are easy to compute and interpret, and they often reduce right skew substantially for strictly positive outcomes like insurance charges.

```
import numpy as np

df['charges_sqrt'] = df['charges']**(1/2)
df['charges_cbrt'] = df['charges']**(1/3)
df['charges_ln'] = np.log(df['charges'])

df[['charges', 'charges_sqrt', 'charges_cbrt', 'charges_ln']].skew()

# Output (example)
# charges             1.515880
# charges_sqrt        0.795863
# charges_cbrt        0.515183
# charges_ln         -0.090098
# dtype: float64
```

Among these simple options, the natural log of *charges* often produces the largest reduction in skewness, bringing it closest to zero. For causal (explanatory) analysis, this can be valuable because it often leads to residuals that are closer to symmetric and variance that is more stable across cases.

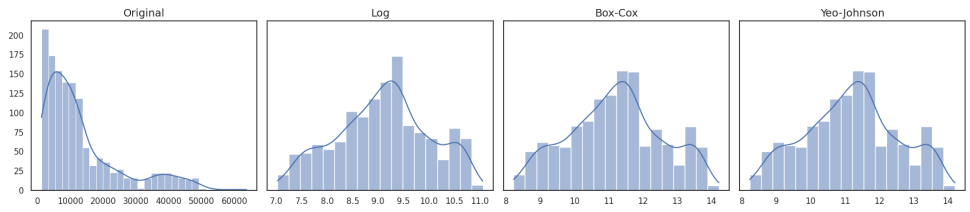
However, not all real-world labels respond well to a single named transformation like log or square root. Two widely used alternatives are *Box-Cox* and *Yeo-Johnson*, which are more flexible because they automatically choose a transformation strength (called *lambda*) based on the data.

- **Box-Cox**: Searches over a family of power transforms and selects the lambda that best reduces skew and stabilizes variance. It requires strictly positive values (no zeros or negatives).
- **Yeo-Johnson**: Similar idea, but it can handle zeros and negative values. This makes it a safer default when you are unsure whether the label contains non-positive values.

In practice, you can think of these as automated, data-driven extensions of the transformations you already know: when lambda is near 0, the transform behaves like a log; when lambda is near 0.5, it behaves like a square root; other lambda values create intermediate or stronger adjustments.

The code below demonstrates both transformations and compares their skewness reduction. Even when the label is already positive (as it is here), it is still useful to show both, because Yeo-Johnson generalizes cleanly to other datasets you may encounter later.

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import PowerTransformer

# --- Prepare label ---
df['charges_ln'] = np.log(df['charges'])

y = df[['charges']]

# Box-Cox (positive values only)
pt_bc = PowerTransformer(method='box-cox', standardize=False)
df['charges_boxcox'] = pt_bc.fit_transform(y)

# Yeo-Johnson (handles zero/negative values)
pt_yj = PowerTransformer(method='yeo-johnson', standardize=False)
df['charges_yj'] = pt_yj.fit_transform(y)

# --- Print skewness comparison ---
skewness = pd.Series({
  'Original': df['charges'].skew(),
  'Log': df['charges_ln'].skew(),
  'Box-Cox': df['charges_boxcox'].skew(),
  'Yeo-Johnson': df['charges_yj'].skew()
})

print(skewness.round(4))

# --- Plot histograms side by side ---
fig, axes = plt.subplots(1, 4, figsize=(18, 4), sharey=True)

plots = [
  ('Original', 'charges'),
  ('Log', 'charges_ln'),
  ('Box-Cox', 'charges_boxcox'),
  ('Yeo-Johnson', 'charges_yj')
]

for ax, (title, col) in zip(axes, plots):
  sns.histplot(df[col], kde=True, ax=ax)
  ax.set_title(title, fontsize=14)
  ax.set_xlabel('')
  ax.set_ylabel('')

plt.tight_layout()
plt.show()
```



At this stage, your goal is to select a transformation that reduces extreme skew and produces a more stable residual pattern after modeling. Although each of the transformations appears to be sufficient for the charges label in this dataset, you will encounter scenarios where Box-Cox

and/or Yeo-Johnson provides superior results to a basic ln() transformation. In the next parts of this chapter, you will see how these univariate improvements connect to the more important target for causal modeling: the normality and structure of the residuals.

Overall model normality refers to the distribution of the *residuals*—the differences between observed and predicted values. Regression assumes that these residuals are approximately normally distributed around zero.

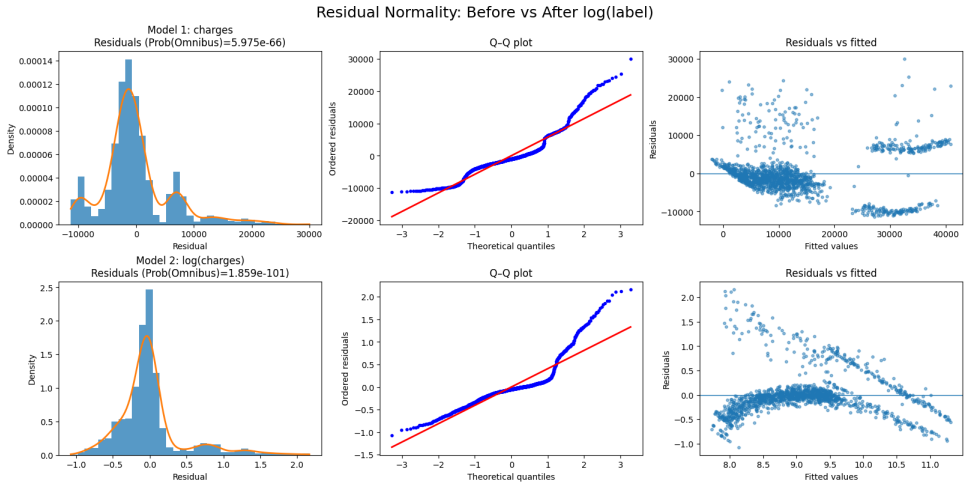
This assumption does not affect point predictions directly. Instead, it affects statistical inference: p-values, confidence intervals, and hypothesis tests on coefficients.

The Omnibus test and its associated p-value (*Prob(Omnibus)*) evaluate whether the residuals deviate significantly from normality based on skewness and kurtosis. Unlike coefficient p-values, a *high* Omnibus p-value indicates that residual normality is not strongly violated. This test appears automatically in the *OLS().summary()* output produced by statsmodels.

Visual inspection often provides more insight than a single test statistic. Residual histograms, Q–Q plots, and residuals-versus-fitted plots help reveal whether errors are centered around zero and whether departures from normality are systematic rather than random.

```
# Insurance residual-normality demo (short version)
# Compares OLS on charges vs OLS on log(charges) and plots 3 diagnostics for each.

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import statsmodels.api as sm
from statsmodels.stats.stattools import omni_normtest
from scipy.stats import probplot, gaussian_kde

# 1) Load + dummy-code
df = pd.read_csv(&quot;/content/drive/MyDrive/Colab Notebooks/data/insurance.csv&quot;)
df = pd.get_dummies(df, columns=df.select_dtypes([&quot;object&quot;]).columns, drop_first=True)

X = sm.add_constant(df.drop(columns=[&quot;charges&quot;]), has_constant=&quot;add&quot;)
y = df[&quot;charges&quot;].astype(float)

# 2) Fit two models
m1 = sm.OLS(y, X).fit()
m2 = sm.OLS(np.log(y), X).fit()

# 3) Show Omnibus values (also appear in .summary())
o1, p1 = omni_normtest(m1.resid)
o2, p2 = omni_normtest(m2.resid)
print(f&quot;Model 1 (charges):      Omnibus={o1:.3f}, Prob(Omnibus)={p1:.6f}&quot;)
print(f&quot;Model 2 (log(charges)): Omnibus={o2:.3f}, Prob(Omnibus)={p2:.6f}&quot;)

# 4) Plot helper: histogram+KDE, Q–Q, residuals vs fitted
def kde_line(r):
  xs = np.linspace(r.min(), r.max(), 300)
  return xs, gaussian_kde(r)(xs)

def row(axs, model, title, p_omni):
  r = np.asarray(model.resid, float)
  f = np.asarray(model.fittedvalues, float)
```

```
    # A) Residual histogram + KDE
    axs[0].hist(r, bins=35, density=True, alpha=0.75)
    xs, ys = kde_line(r)
    axs[0].plot(xs, ys, linewidth=2)
    axs[0].set_title(f&quot;{title}\nResiduals (Prob(Omnibus)={p_omni:.4g})&quot;)
    axs[0].set_xlabel(&quot;Residual&quot;)
    axs[0].set_ylabel(&quot;Density&quot;)

    # B) Q–Q plot
    probplot(r, dist=&quot;norm&quot;, plot=axs[1])
    axs[1].get_lines()[0].set_markersize(3)
    axs[1].get_lines()[1].set_linewidth(2)
    axs[1].set_title(&quot;Q–Q plot&quot;)
    axs[1].set_xlabel(&quot;Theoretical quantiles&quot;)
    axs[1].set_ylabel(&quot;Ordered residuals&quot;)

    # C) Residuals vs fitted
    axs[2].scatter(f, r, s=10, alpha=0.5)
    axs[2].axhline(0, linewidth=1)
    axs[2].set_title(&quot;Residuals vs fitted&quot;)
    axs[2].set_xlabel(&quot;Fitted values&quot;)
    axs[2].set_ylabel(&quot;Residuals&quot;)

# 5) Composite figure (2 rows × 3 columns)
fig, ax = plt.subplots(2, 3, figsize=(16, 8), constrained_layout=True)
row(ax[0], m1, &quot;Model 1: charges&quot;, p1)
row(ax[1], m2, &quot;Model 2: log(charges)&quot;, p2)
plt.suptitle(&quot;Residual Normality: Before vs After log(label)&quot;, fontsize=18)
plt.show()

# Output:
# Model 1 (charges):      Omnibus=300.366, Prob(Omnibus)=0.000000
# Model 2 (log(charges)): Omnibus=463.882, Prob(Omnibus)=0.000000
```



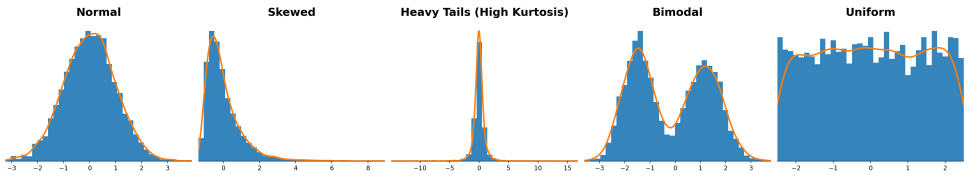
In the example above, we compared an untransformed model with a model that applies a logarithmic transformation to the label. The residual histogram and Q–Q plot typically become more symmetric after the log transform, which can make inference more stable. However, the Omnibus p-value may still be very small, reminding us that residual normality is influenced by more than label skew alone.

For causal (explanatory) modeling, the key question is not whether the residuals look perfectly normal, but whether departures from normality are severe enough to undermine coefficient tests and confidence intervals. In practice, strong non-normality often indicates that other

assumptions are violated at the same time—especially linearity or homoscedasticity—so fixing normality in isolation is rarely the best strategy.

Rather than treating normality as a pass/fail condition, it is more useful to view it as a diagnostic signal that points toward underlying modeling issues such as non-linearity, missing structure, subgroup effects, or unstable variance. Different distributional patterns suggest different causes—and therefore different responses.

To summarize, there are many reasons that non-normality exists and many fixes. We will cover the rest of them in the remainder of this chapter, but it may be helpful to see a summary of these issues here:



Common distributions

Table 10.2
Normality Diagnostics: Patterns, Causes, and Modeling Implications

Observed Pattern	What It Usually Indicates	Diagnostic Signal	Typical Adjustment	Threat to Causal Modeling?	Threat to Predictive Modeling?
Approximately normal	Errors are symmetric and stable	Residual histogram and Q–Q plot align well	No action needed	No	No
Skewed distribution	Extreme values dominate variability	Long tail in residual histogram; curvature in Q–Q plot	Label transformation (log, Box–Cox, Yeo–Johnson)	Yes — affects p-values and confidence intervals	Usually no, unless extreme
Heavy tails (high kurtosis)	Outliers occur more frequently than expected	Large deviations at the ends of the Q–Q plot	Use caution with inference; consider robust approaches	Yes — increases risk of misleading significance	Often no, if predictions are validated
Multimodal distribution	Multiple subpopulations or missing structure	Multiple peaks in the residual or label distribution	Add categorical variables or interaction terms	Yes — coefficients may conflate distinct groups	Sometimes — depends on stability across groups

Observed Pattern	What It Usually Indicates	Diagnostic Signal	Typical Adjustment	Threat to Causal Modeling?	Threat to Predictive Modeling?
Discrete or clumped values	Counts, rounding, or bounded outcomes	Stacked residuals or visible banding	Acknowledge approximation limits; consider alternative models	Yes — standard errors may be unreliable	Often no for rough prediction
Conditional non-normality	Error behavior changes across prediction ranges	Residual spread varies with fitted values	Revisit linearity or homoscedasticity	Yes — violates assumptions underlying inference	Sometimes — may hurt generalization

This table reinforces a central principle of regression diagnostics: departures from normality are *signals*, not automatic failures. The appropriate response depends on the observed pattern, the modeling objective, and whether the analysis prioritizes explanation or prediction.

Normality diagnostics matter most for *explanatory (causal) analysis*, where valid p-values and confidence intervals are essential. For prediction-focused modeling, modest departures from normality are common and often acceptable as long as predictive performance is evaluated on unseen data and uncertainty is communicated appropriately.

10.4Multicollinearity

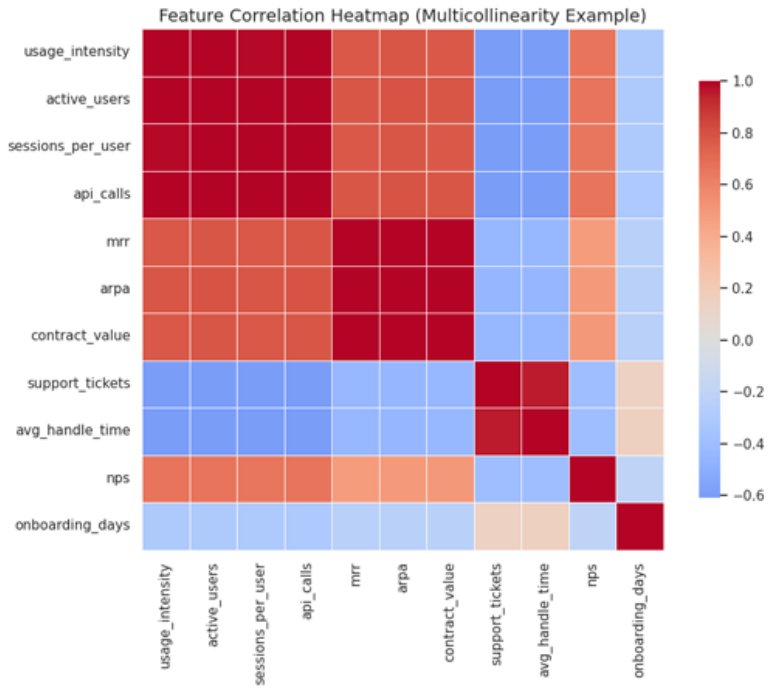
Multicollinearity The presence of strong correlations among independent variables. Multicollinearity occurs when two or more features convey overlapping information. In multiple linear regression, this overlap makes it difficult to isolate the unique effect of each feature, which is exactly the kind of interpretation challenge that matters most in *causal (explanatory)* modeling.

One of the most intuitive ways to detect potential multicollinearity is to examine a correlation heatmap of the predictor variables. Each cell in the heatmap represents the pairwise correlation between two features, with darker colors indicating stronger positive or negative relationships.

In the heatmap below, several groups of features exhibit extremely high correlations with one another. For example, metrics related to product usage—such as *usage_intensity*, *active_users*, *sessions_per_user*, and *api_calls*—move together almost perfectly. This indicates that these features are capturing very similar underlying behavior.

These values come from a SaaS customer churn dataset designed to understand why customers stop using a subscription-based software product. In this business context, usage-related features

such as *active_users*, *sessions_per_user*, *api_calls*, and *usage_intensity* all measure closely related aspects of customer engagement, which explains why they exhibit extremely high correlations.



Strong pairwise correlations are an early warning sign, but *correlation alone does not imply multicollinearity*; true multicollinearity exists only when a feature can be explained by a combination of all other features in the model.

High correlations like these signal potential multicollinearity concerns for explanatory modeling. If all of these features were included in the same regression, the model would struggle to isolate the unique effect of each variable because the features rarely vary independently. However, correlation is a bivariate measure that does not account for the combined influence of all other predictors, which is why a more formal diagnostic is required.

Multicollinearity is primarily a problem for *interpretation*, not prediction. When features are highly correlated, the model can still generate accurate in-sample predictions, but the coefficient estimates become unstable: small changes in the data can cause large changes in coefficient values and even coefficient signs.

This instability shows up as inflated standard errors, weaker or inconsistent *t*-statistics, and misleading p-values. In other words, multicollinearity makes it difficult to answer the

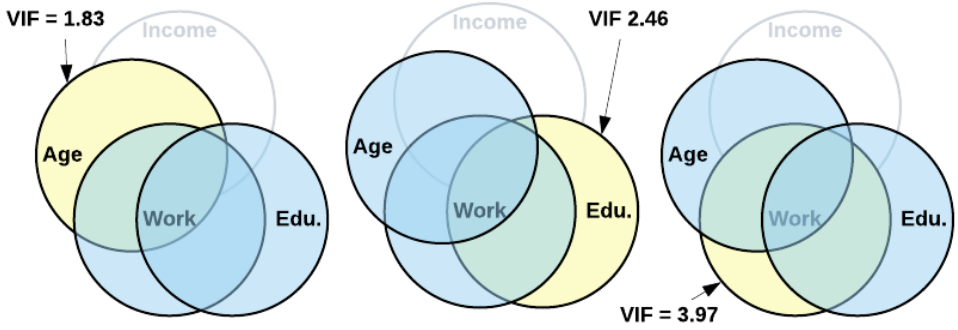
explanatory question, “What is the unique effect of this feature, holding the others constant?” because the features do not truly vary independently.

Statsmodels reports the *Condition Number (Cond. No.)* as a rough warning signal for numerical dependence in the design matrix. Large values can occur when predictors have very different scales, when predictors are nearly linearly dependent, or when both issues occur at the same time. However, the condition number does not identify which specific features are responsible for the overlap, and it should not be treated as a substitute for feature-level diagnostics such as VIF.

To pinpoint feature-level multicollinearity, we use *Variance Inflation Factor (VIF)*, which quantifies how much a coefficient’s variance is inflated because the feature can be predicted using the other features in the model.

Conceptually, VIF asks a simple question: “If I tried to predict this feature using all the other features, how well could I do?” A high VIF indicates that the feature contributes little independent information and is therefore problematic for explanatory interpretation.

If a feature can be predicted very well by the others (high R^2 in that auxiliary regression), then it contributes little independent information, and its VIF will be large.



To understand how VIF works, we can calculate it manually by fitting a separate regression for each feature against all remaining features and extracting the R^2. Then we convert that R^2 into VIF using $VIF = 1 / (1 - R^2)$.

This manual loop is instructional. It reinforces that VIF is not a mysterious statistic; it is derived directly from a familiar regression idea: “How well can one feature be explained by the others?”

```
import pandas as pd
import statsmodels.api as sm
```

```
# Use original dataframe (already dummy-coded elsewhere if needed)
X = df.drop(columns=[&amp;quot;charges&amp;quot;]).copy()

# Convert boolean columns to integers
X[X.select_dtypes(bool).columns] = X.select_dtypes(bool).astype(int)

# Add constant
X = sm.add_constant(X, has_constant=&amp;quot;add&amp;quot;)

# DataFrame to store results
df_vif = pd.DataFrame(columns=[&amp;quot;VIF&amp;quot;])

# Loop through each feature (excluding the constant)
for col in X.columns:
  if col == &amp;quot;const&amp;quot;:
    continue

  y_aux = X[col]
  X_aux = X.drop(columns=[col])

  r_squared = sm.OLS(y_aux, X_aux).fit().rsquared
  df_vif.loc[col] = [1 / (1 - r_squared)]

df_vif.sort_values(by=&amp;quot;VIF&amp;quot;, ascending=False)
```

	VIF
region_southeast	1.652230
region_southwest	1.529411
region_northwest	1.518823
bmi	1.106630
age	1.016822
smoker_yes	1.012074
sex_male	1.008900
children	1.004011

VIF values are commonly interpreted using approximate thresholds:

- VIF < 3: Little to no multicollinearity (ideal for explanatory modeling).
- VIF between 3 and 5: Moderate multicollinearity (often acceptable with careful interpretation).
- VIF > 10: High multicollinearity (problematic for coefficient interpretation).

Based on this, our results above indicate that none of the features have multicollinearity problems. This is expected since none of the features are theoretically similar.

Also, these thresholds above are guidelines, not rules. Whether a VIF value is “too high” depends on your goal. If the goal is causal or explanatory interpretation, high VIF can seriously undermine coefficient-level conclusions. If the goal is prediction, high VIF is often tolerable as long as predictive performance is validated appropriately.

A common student misunderstanding is to assume that “high multicollinearity means the model is wrong.” A more accurate mental model is: multicollinearity mostly means “the model cannot confidently assign credit to one feature versus another,” even though the group of correlated features may still predict the label well.

Automated VIF Calculation

In practice, we rarely compute VIF manually. A standard workflow is to compute VIF using a built-in function and then investigate the specific features with the highest VIF values.

The following code produces the same VIF information more efficiently. Notice that the logic is identical: the function computes an auxiliary regression for each column and then applies the same VIF formula.

```
import pandas as pd
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor

# Use original dataframe
X = df.drop(columns=[&amp;quot;charges&amp;quot;]).copy()

# Convert boolean columns to integers
X[X.select_dtypes(bool).columns] = X.select_dtypes(bool).astype(int)

# Add constant
X = sm.add_constant(X, has_constant=&amp;quot;add&amp;quot;)

vif_df = pd.DataFrame({
  &amp;quot;feature&amp;quot;: X.columns,
  &amp;quot;VIF&amp;quot;: [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
})

# Optional: drop constant from reporting
vif_df = vif_df[vif_df[&amp;quot;feature&amp;quot;] != &amp;quot;const&amp;quot;]

vif_df.sort_values(by=&amp;quot;VIF&amp;quot;, ascending=False)
```

	feature	VIF
7	region_southeast	1.652230
8	region_southwest	1.529411
6	region_northwest	1.518823
2	bmi	1.106630
1	age	1.016822
5	smoker_yes	1.012074
4	sex_male	1.008900
3	children	1.004011

If you prefer, you can compute VIF from a scikit-learn style matrix as long as you pass a purely numeric feature matrix (dummy codes included) into the VIF function. The key requirement is that the matrix contains predictors only (no label column).

Once multicollinearity is identified, the typical next steps include removing redundant features, combining correlated features into a single construct, or choosing a reference coding approach for categorical variables to avoid perfect redundancy. In causal (explanatory) work, the goal is not to maximize the number of predictors, but to ensure that each coefficient corresponds to a distinct and interpretable concept.

Multicollinearity affects how confidently we can interpret coefficients, but it does not imply time dependence or sequencing in the data. That concern is addressed by the next assumption: autocorrelation.

Finally, the table below summarizes various patterns of multicollinearity, where you might see them, what it often means, and what you should do about it:

Table 10.3
How Multicollinearity Shows Up and What It Means

What You Observe	Where You See It	What It Often Means	Common Next Step
Large VIF values for a small set of features	VIF table	Those features contain overlapping information	Remove one feature or consolidate features
Coefficients change a lot when you add/remove a correlated predictor	Model output across runs	Coefficient estimates are unstable due to redundancy	Use VIF to identify redundancy; prioritize domain meaning
High standard errors and weak t-tests despite good overall fit	Coefficient table in OLS summary	Model predicts well but cannot attribute effects cleanly	Treat inference cautiously; consider simplifying predictors
Perfect redundancy warnings (singular matrix, extreme condition number)	Model notes / diagnostics	Dummy trap or duplicated information	Drop a reference category; remove redundant columns

10.5Autocorrelation

Autocorrelation Correlation between observations caused by time ordering or sequence dependence.
Autocorrelation occurs when residuals from one observation are correlated with residuals from nearby observations, typically because the data is ordered in time. This matters for causal

(explanatory) regression because autocorrelation can make *standard errors* misleading, which undermines confidence intervals and hypothesis tests.

This assumption applies primarily to *time-series* or sequential data. In cross-sectional datasets—where each row represents an independent entity—autocorrelation is usually not a concern. For the causal inference focus of this chapter, the key takeaway is simple: you must check autocorrelation when your observations have a meaningful order (usually time), because that order can invalidate standard inference.

It is important to distinguish autocorrelation from multicollinearity. Autocorrelation concerns relationships *between observations* (rows), not relationships between features (columns). Multicollinearity threatens your ability to interpret coefficients; autocorrelation threatens the validity of your standard errors and tests.

For example, using *age in 2018* and *age in 2020* as separate features creates multicollinearity, not autocorrelation. Autocorrelation arises when residuals are linked across time (for example, today's error is related to yesterday's error).

The *Durbin-Watson (DW) statistic* is the most common test for first-order autocorrelation in regression residuals. It ranges from 0 to 4:

- DW ≈ 2: No autocorrelation (ideal for inference).
- DW < 2: Positive autocorrelation.
- DW > 2: Negative autocorrelation.

Positive autocorrelation means that errors tend to repeat in the same direction over time, while negative autocorrelation indicates alternating error patterns. Both violate the OLS assumption that residuals are independent across observations.

When autocorrelation is present, coefficient estimates can remain unbiased under common conditions, but standard errors are incorrect. This creates false confidence: p-values may look "significant" and confidence intervals may look "tight" even when inference is not valid. Because this chapter emphasizes causal (explanatory) interpretation, autocorrelation is a high-priority diagnostic whenever the data is time-ordered.

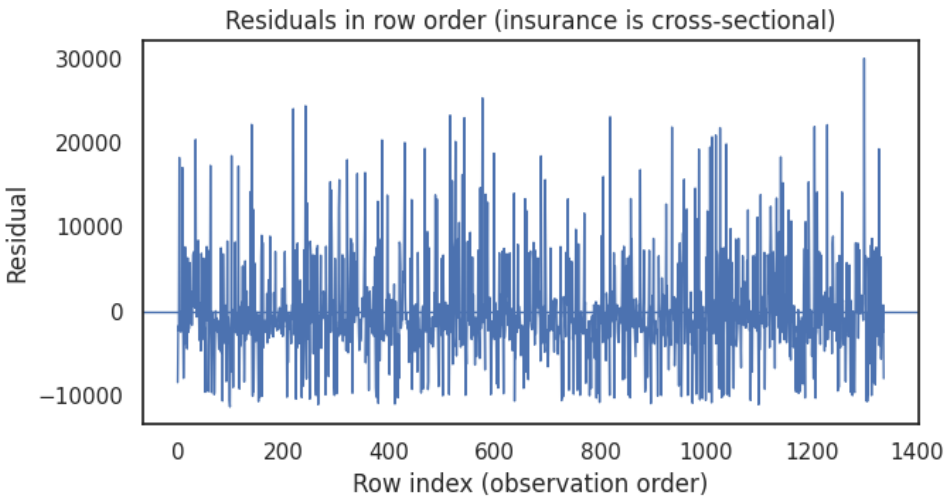
Illustrating Autocorrelation with a Cross-Sectional Dataset

The insurance dataset is cross-sectional, so we do not expect autocorrelation. However, it is still useful to practice the diagnostic workflow. The key idea is that autocorrelation is about whether

residuals are related across an observation order. For time-series data, that order is time. For cross-sectional data, the row order is usually arbitrary, so a good diagnostic should show no systematic pattern.

We will fit an OLS model, report the Durbin-Watson statistic, and then visualize whether residuals show any run-like pattern when plotted in row order. Finally, we will compute the lag-1 correlation of residuals as an intuitive check. In a causal workflow, this step is a quick way to confirm that independence across observations is a reasonable assumption before you proceed to functional-form diagnostics such as linearity.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
from statsmodels.stats.stattools import durbin_watson

# 1) Load + dummy-code (cross-sectional)
df = pd.read_csv(&quot;/content/drive/MyDrive/Colab Notebooks/data/insurance.csv&quot;)
df = pd.get_dummies(df, columns=df.select_dtypes([&quot;object&quot;]).columns, drop_first=True)

# 2) Fit OLS
y = df[&quot;charges&quot;].astype(float)
X = sm.add_constant(df.drop(columns=[&quot;charges&quot;]), has_constant=&quot;add&quot;)
m = sm.OLS(y, X).fit()

# 3) Durbin-Watson (also appears in m.summary())
dw = durbin_watson(m.resid)
print(f&quot;Durbin-Watson: {dw:.3f}&quot;)

# 4) Residuals in observation order (should look patternless for cross-sectional data)
r = np.asarray(m.resid, float)

plt.figure(figsize=(7, 3.8))
plt.plot(r, linewidth=1)
plt.axhline(0, linewidth=1)
plt.xlabel(&quot;Row index (observation order)&quot;)
plt.ylabel(&quot;Residual&quot;)
plt.title(&quot;Residuals in row order (insurance is cross-sectional)&quot;)
plt.tight_layout()
plt.show()

# 5) Simple lag-1 correlation as an intuitive check
lag1_corr = np.corrcoef(r[1:], r[:-1])[0, 1]
print(f&quot;Lag-1 residual correlation: {lag1_corr:.4f}&quot;)

# Output (example format):
# Durbin-Watson: 2.088
# Lag-1 residual correlation: -0.0456
```



In the output printed above, the Durbin-Watson statistic is approximately 2, which is the “no autocorrelation” benchmark. In the figure, the residuals also do not show long runs of mostly-positive or mostly-negative values. Taken together, those two signals support the conclusion that autocorrelation is not a concern for this cross-sectional dataset.

The lag-1 residual correlation provides an intuitive complement to Durbin–Watson. In the printed output above, the lag-1 correlation is close to zero, meaning each residual is not meaningfully related to the residual immediately before it in row order. For causal (explanatory) regression, this is what you want to see: if errors were serially dependent, your standard errors and p-values could be overly optimistic.

Important limitation of this demonstration
Because the insurance dataset is not time-ordered, the row index is not a meaningful sequence. For time-series data, you would plot residuals against time (or observation order by time) and interpret Durbin-Watson in that context.

Autocorrelation is most critical in forecasting problems, sensor data, financial time series, and any setting where observations are ordered and influence one another. In those settings, autocorrelation is a direct threat to inference if you attempt to use standard OLS standard errors.

In those cases, ordinary least squares may be inappropriate, and specialized approaches such as generalized least squares, Newey–West (HAC) robust standard errors, or time-series models are often required.

Having confirmed that observation independence is reasonable for this dataset, we can next evaluate whether the relationship between features and the label is appropriately modeled as

linear in the model space.

10.6Linearity

The linearity assumption states that the relationship between each feature and the label can be reasonably approximated by a straight line, holding other variables constant. For causal (explanatory) modeling, this assumption is especially important because a mis-specified functional form can bias coefficient estimates and lead to misleading conclusions about effect sizes.

This assumption applies to the *functional form* of the relationship—not to the raw data values themselves. In other words, the question is not whether the raw scatterplot looks like a straight line, but whether the model structure correctly represents how the expected outcome changes with each predictor.

Linearity does not require that features themselves be normally distributed, nor that the relationship be visually straight in raw scatterplots. Instead, it requires that the expected value of the label changes linearly with each feature *in the model space* after any transformations you choose to apply.

A practical way to assess linearity is to look for systematic structure in errors. When linearity is violated, residuals often show curved or wave-like patterns when plotted against fitted values or against a specific feature. In causal modeling, these patterns are a warning sign that the coefficient estimates may be describing the wrong relationship.

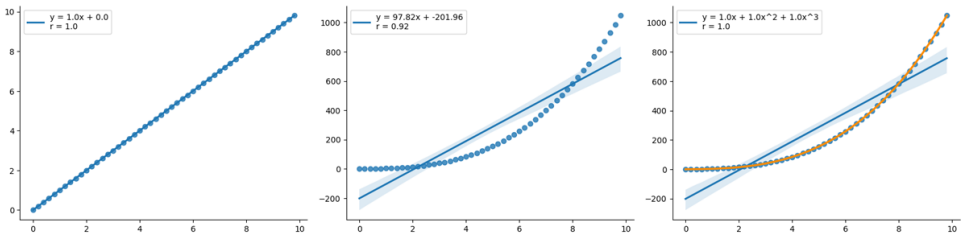


Figure 10.1: Examples of Linear and Non-Linear Correlations

The left plot shows a perfect linear relationship, which is ideal for multiple linear regression. The middle plot shows a curved relationship, where a straight line would systematically misrepresent the pattern. The right plot demonstrates how polynomial terms (such as x^2 or x^3) can restore linearity in the model space.

When a feature violates the linearity assumption, the solution is often not to abandon regression, but to revise the model's functional form. Common approaches include logarithmic, square root, exponential, or polynomial transformations.

These transformations change the *scale* of the feature so that the relationship becomes approximately linear in the transformed space. For causal inference, the goal is not simply a better fit, but a functional form that makes coefficient interpretation more defensible.

Linearity is most critical when the goal is relationship *interpretation*. If coefficients are used to explain cause-and-effect relationships, violations of linearity can lead to misleading conclusions because the model is estimating the wrong functional form.

For prediction tasks, mild nonlinearity can be less damaging, and many machine learning models capture nonlinear patterns automatically. In contrast, multiple linear regression requires the analyst to specify the functional form explicitly, which is why linearity diagnostics play such a central role in an inference-focused workflow.

From a deployment perspective, linearity matters because transformations applied during training must be applied identically during inference. Any transformation used to correct nonlinearity becomes part of the production pipeline. Even though this chapter emphasizes causal interpretation, this reminder helps reinforce that diagnostic fixes have implementation consequences.

In this section, we will use the insurance dataset to (1) visualize potential nonlinear patterns, (2) fit a baseline model, (3) add nonlinear terms (polynomials and logs), and (4) compare whether residual patterns become more random. The emphasis throughout is diagnostics-driven refinement for more trustworthy coefficient interpretation.

Table 10.4
Linearity Diagnostics: Common Signals and Typical Responses

Diagnostic Signal	What It Suggests	Typical Response
Curved pattern in residuals vs fitted	Model is missing a nonlinear functional form	Add polynomial terms or transform the relevant feature (log, square root)
Residuals systematically increase for large feature values	Relationship steepens or flattens at extremes	Try a log transform, quadratic term, or piecewise modeling
Fan shape in residuals vs fitted	Variance changes with prediction level (often overlaps with homoscedasticity)	Consider transforming the label, transforming features, or using robust standard errors

Why We Tried Age² and log(BMI)

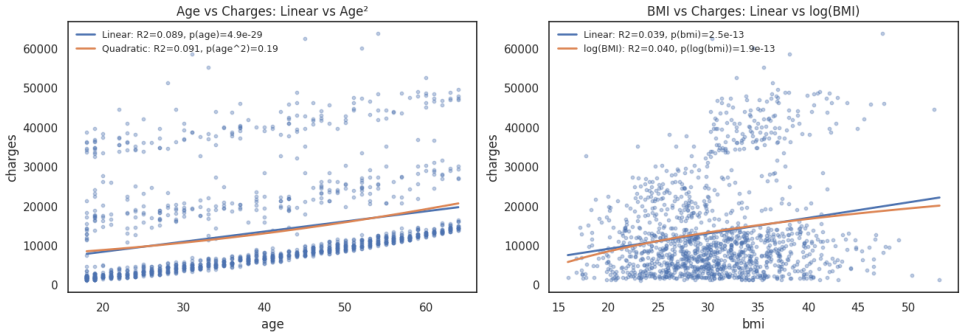
Before adding nonlinear terms to a full multiple regression model, it helps to look at simple bivariate relationships. If a feature has an obviously curved relationship with *charges*, a straight-line term may systematically miss that pattern, which often shows up later as curved structure in residual plots.

In the insurance dataset, *age* often shows accelerating changes in *charges* at older ages, which makes a quadratic term (*age²*) a reasonable first attempt. Likewise, *bmi* often relates to *charges* in a way that becomes steeper at higher BMI values, which makes a logarithmic transform (*log(bmi)*) a reasonable first attempt.

The code below produces two scatterplots (Age vs Charges, BMI vs Charges). Each plot overlays a linear trendline and a curved alternative, and annotates each trendline with its R^2 and the key term's *p-value*, so you can see whether the curved form fits better. In an inference-focused workflow, this is a preliminary justification step before you confirm improvements through residual diagnostics.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm

# Load insurance (update path if needed)
df = pd.read_csv(&quot;/content/drive/MyDrive/Colab Notebooks/data/insurance.csv&quot;)
df = pd.get_dummies(df, columns=df.select_dtypes([&quot;object&quot;]).columns, drop_first=True)

y = df[&quot;charges&quot;].astype(float)

def fit_lin(x, y):
  X = sm.add_constant(x, has_constant=&quot;add&quot;)
  return sm.OLS(y, X).fit()

def fit_quad(x, y):
  X = pd.DataFrame({&quot;x&quot;: x, &quot;x2&quot;: x**2})
  X = sm.add_constant(X, has_constant=&quot;add&quot;)
  return sm.OLS(y, X).fit()

def fit_logx(x, y):
  x_ln = np.log(x)
  X = sm.add_constant(x_ln, has_constant=&quot;add&quot;)
  return sm.OLS(y, X).fit()

def line_from_model(x_grid, model, kind):
  if kind == &quot;lin&quot;:
    Xg = sm.add_constant(x_grid, has_constant=&quot;add&quot;)
    return model.predict(Xg)
  if kind == &quot;quad&quot;:
    Xg = pd.DataFrame({&quot;x&quot;: x_grid, &quot;x2&quot;: x_grid**2})
    Xg = sm.add_constant(Xg, has_constant=&quot;add&quot;)
    return model.predict(Xg)
  if kind == &quot;logx&quot;:
    Xg = sm.add_constant(np.log(x_grid), has_constant=&quot;add&quot;)
    return model.predict(Xg)

# 1) Age vs charges (linear vs quadratic)
x_age = df[&quot;age&quot;].astype(float)
```

```
m_age_lin = fit_lin(x_age, y)
m_age_quad = fit_quad(x_age, y)

age_grid = np.linspace(x_age.min(), x_age.max(), 250)
y_age_lin = line_from_model(age_grid, m_age_lin, &amp;quot;lin&amp;quot;)
y_age_quad = line_from_model(age_grid, m_age_quad, &amp;quot;quad&amp;quot;)

# 2) BMI vs charges (linear vs log(BMI))
x_bmi = df[&amp;quot;bmi&amp;quot;].astype(float)
m_bmi_lin = fit_lin(x_bmi, y)
m_bmi_log = fit_logx(x_bmi, y)

bmi_grid = np.linspace(max(1e-6, x_bmi.min()), x_bmi.max(), 250)
y_bmi_lin = line_from_model(bmi_grid, m_bmi_lin, &amp;quot;lin&amp;quot;)
y_bmi_log = line_from_model(bmi_grid, m_bmi_log, &amp;quot;logx&amp;quot;)

fig, ax = plt.subplots(1, 2, figsize=(13, 4.5), constrained_layout=True)

ax[0].scatter(x_age, y, s=10, alpha=0.35)
ax[0].plot(age_grid, y_age_lin, linewidth=2, label=f&amp;quot;Linear: R2={m_age_lin.rsquared:.3f}, p(age)=
{m_age_lin.pvalues.iloc[1]:.2g}&amp;quot;)
ax[0].plot(age_grid, y_age_quad, linewidth=2, label=f&amp;quot;Quadratic: R2={m_age_quad.rsquared:.3f},
p(age^2)={m_age_quad.pvalues['x2']:.2g}&amp;quot;)
ax[0].set_title(&amp;quot;Age vs Charges: Linear vs Age²&amp;quot;)
ax[0].set_xlabel(&amp;quot;age&amp;quot;)
ax[0].set_ylabel(&amp;quot;charges&amp;quot;)
ax[0].legend(frameon=False, fontsize=9)

ax[1].scatter(x_bmi, y, s=10, alpha=0.35)
ax[1].plot(bmi_grid, y_bmi_lin, linewidth=2, label=f&amp;quot;Linear: R2={m_bmi_lin.rsquared:.3f}, p(bmi)=
{m_bmi_lin.pvalues.iloc[1]:.2g}&amp;quot;)
ax[1].plot(bmi_grid, y_bmi_log, linewidth=2, label=f&amp;quot;log(BMI): R2={m_bmi_log.rsquared:.3f},
p(log(bmi))={m_bmi_log.pvalues.iloc[1]:.2g}&amp;quot;)
ax[1].set_title(&amp;quot;BMI vs Charges: Linear vs log(BMI)&amp;quot;)
ax[1].set_xlabel(&amp;quot;bmi&amp;quot;)
ax[1].set_ylabel(&amp;quot;charges&amp;quot;)
ax[1].legend(frameon=False, fontsize=9)

plt.show()
```



In the Age vs Charges panel, compare the two annotated R^2 values shown in the legend. If the quadratic model's R^2 is meaningfully higher than the linear model's R^2 and the *p-value* for age^2 is small, that is evidence that a curved relationship is present and that a straight-line age term alone is missing structure. For causal interpretation, this matters because the "effect of age" is not constant across the age range when the relationship is curved.

In the BMI vs Charges panel, compare the linear trendline to the *log(BMI)* trendline. If the *log(BMI)* model shows higher R^2 and a small *p-value* for the log term, that supports trying a log transform in the multivariate model. Even if the improvement is modest, this bivariate view

provides a concrete rationale for the transformation before you evaluate residual diagnostics in the full regression.

Interpreting Models with Both Linear and Squared Terms

When you add a squared term (such as *age²*) to a model that already includes the linear term (such as *age*), both terms work *together* to capture the non-linear relationship. This is different from replacing the linear term with a squared term—you keep both because they serve different purposes.

The linear term (e.g., *age*) captures the base effect or starting slope of the relationship. The squared term (e.g., *age²*) captures how that effect changes as the feature value increases—it models the curvature. Together, they allow the model to represent relationships where the effect of a feature depends on its current value.

For example, if a model includes both *age* and *age²*, the effect of increasing age by one year is not constant. Instead, the marginal effect depends on the current age value. At younger ages, the effect might be smaller, while at older ages, the effect might be larger (or vice versa, depending on the coefficient signs). This is why both terms are needed: the linear term provides the baseline, and the squared term adjusts that baseline based on the feature's value.

Importantly, you cannot interpret the linear and squared coefficients separately. They must be interpreted together because they jointly determine how the feature influences the outcome. The coefficient for *age* alone does not tell you the effect of age—you need both coefficients to understand the relationship at any given age value.

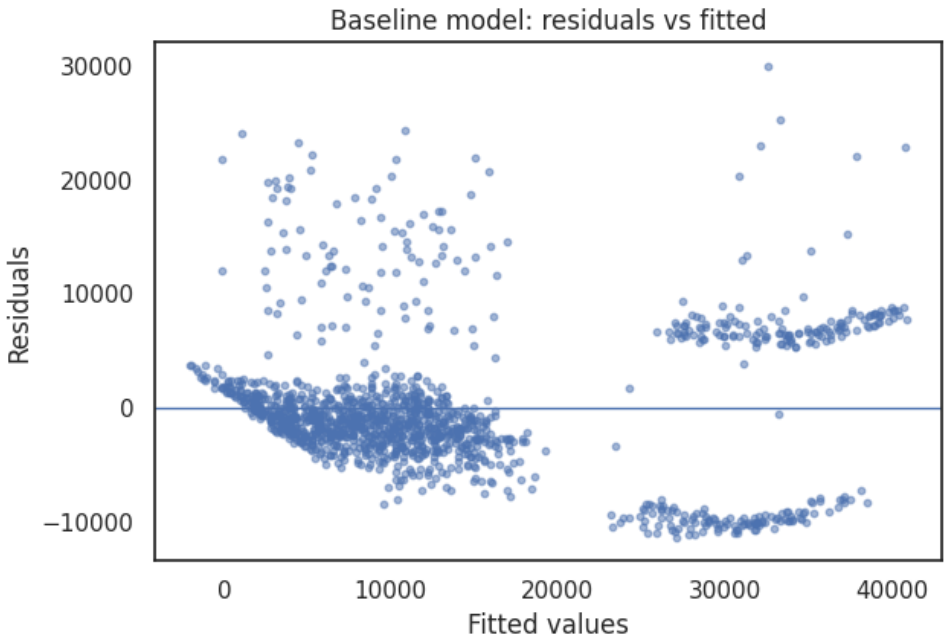
This also explains why polynomial terms and their base features will have high variance inflation factors (VIF) with each other—they are naturally correlated. However, this high VIF is *expected and acceptable* for polynomial terms because both are necessary to capture the non-linear relationship. Removing one would eliminate the ability to model the curvature, which defeats the purpose of adding the polynomial term in the first place.

For causal (explanatory) modeling, understanding that marginal effects depend on current values is crucial for accurate interpretation. When you report that "increasing age by one year increases charges by X dollars," that statement is only valid at a specific age value. The effect will be different at age 30 versus age 60, which is exactly what polynomial terms allow the model to capture.

Linearity Demo in Python (Insurance Dataset)

We begin by fitting a baseline OLS model and then plotting residuals versus fitted values. For causal (explanatory) modeling, the purpose of this plot is not “to get the best-looking scatter,” but to check whether the model is systematically mis-specifying the functional form. When residuals show clear curvature, waves, or structured clustering, the model may be assigning biased or misleading effect estimates to one or more predictors.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm

df = pd.read_csv(&amp;quot;/content/drive/MyDrive/Colab Notebooks/data/insurance.csv&amp;quot;)
df = pd.get_dummies(df, columns=df.select_dtypes([&amp;quot;object&amp;quot;]).columns, drop_first=True)

y = df[&amp;quot;charges&amp;quot;].astype(float)
X = sm.add_constant(df.drop(columns=[&amp;quot;charges&amp;quot;]), has_constant=&amp;quot;add&amp;quot;)
X[X.select_dtypes(bool).columns] = X.select_dtypes(bool).astype(int)

m_base = sm.OLS(y, X).fit()

fitted = np.asarray(m_base.fittedvalues, float)
resid = np.asarray(m_base.resid, float)

plt.figure(figsize=(6.5, 4.5))
plt.scatter(fitted, resid, s=10, alpha=0.5)
plt.axhline(0, linewidth=1)
plt.xlabel(&amp;quot;Fitted values&amp;quot;)
plt.ylabel(&amp;quot;Residuals&amp;quot;)
plt.title(&amp;quot;Baseline model: residuals vs fitted&amp;quot;)
plt.tight_layout()
plt.show()

# Optional:
# print(m_base.rsquared, m_base.rsquared_adj)
```



What You Are Seeing in the Baseline Residual Plot

In the baseline residuals versus fitted plot above, the horizontal line at zero represents “perfect” predictions (no error). Each point shows one observation’s error: points above zero are underpredictions (actual charges are higher than predicted), and points below zero are overpredictions.

For the linearity assumption, the key question is whether residuals look like random noise around zero across the fitted-value range. In this dataset, the residuals form visible clusters and non-random structure rather than a single, pattern-free cloud. Some of that structure is driven by strong subgroup separation (especially smokers versus non-smokers), but curvature within clusters can also indicate that one or more numeric relationships are not well captured by a straight-line functional form.

In regression output, *fitted values* (sometimes called predicted values) are the model’s estimated values of the label for each observation, based on the learned coefficients and the observed feature values. Each fitted value represents what the model expects the outcome to be for that row, given its inputs.

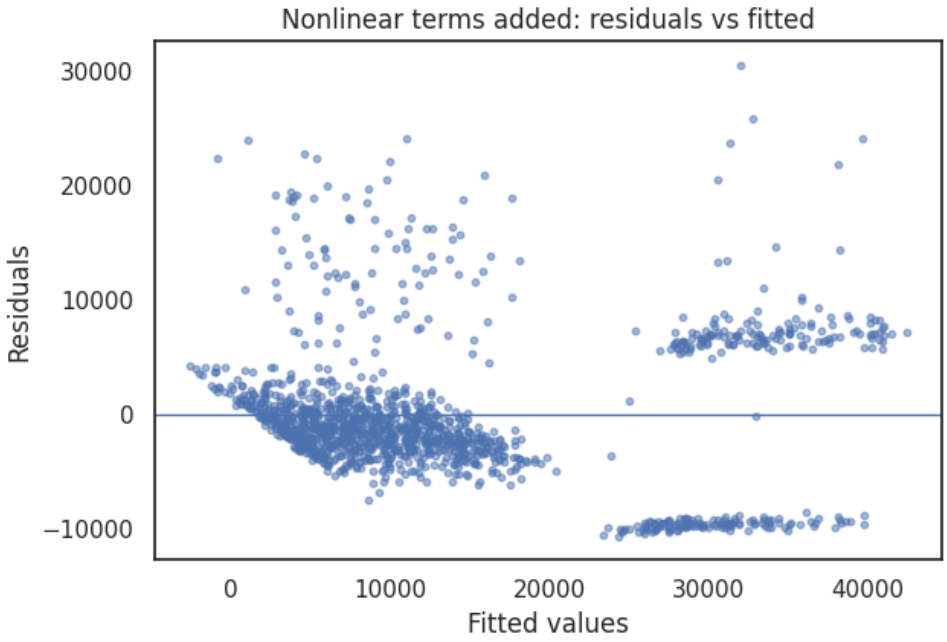
Adding Non-Linear Terms

Next, we add a small number of targeted nonlinear terms. The goal is not to add complexity everywhere, but to adjust specific relationships when diagnostics suggest curvature. Two common approaches are (1) polynomial terms (such as age^2) and (2) log transforms (such as *log(bmi)*). In a causal workflow, these terms are justified when they reduce systematic residual structure and make the coefficient interpretation closer to the true relationship.

A useful rule of thumb is to start simple: add one nonlinear term at a time, refit, and then check whether residual patterns become more random. If residual structure remains, that suggests the model is still missing key functional form details.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm

df = pd.read_csv(&amp;quot;/content/drive/MyDrive/Colab Notebooks/data/insurance.csv&amp;quot;)
df = pd.get_dummies(df, columns=df.select_dtypes([&amp;quot;object&amp;quot;]).columns, drop_first=True)

# Baseline model
y = df[&amp;quot;charges&amp;quot;].astype(float)
X = sm.add_constant(df.drop(columns=[&amp;quot;charges&amp;quot;]), has_constant=&amp;quot;add&amp;quot;)
X[X.select_dtypes(bool).columns] = X.select_dtypes(bool).astype(int)
m_base = sm.OLS(y, X).fit()
```

```
# Add targeted nonlinear terms
df_nl = df.copy()
df_nl[&amp;quot;age_sq&amp;quot;] = df_nl[&amp;quot;age&amp;quot;] ** 2
df_nl[&amp;quot;bmi_ln&amp;quot;] = np.log(df_nl[&amp;quot;bmi&amp;quot;])

y2 = df_nl[&amp;quot;charges&amp;quot;].astype(float)
X2 = sm.add_constant(df_nl.drop(columns=[&amp;quot;charges&amp;quot;]), has_constant=&amp;quot;add&amp;quot;)
X2[X2.select_dtypes(bool).columns] = X2.select_dtypes(bool).astype(int)
m_nl = sm.OLS(y2, X2).fit()

print(f&amp;quot;Baseline:  R2={m_base.rsquared:.4f}, Adj R2={m_base.rsquared_adj:.4f}&amp;quot;)
print(f&amp;quot;Nonlinear: R2={m_nl.rsquared:.4f}, Adj R2={m_nl.rsquared_adj:.4f}&amp;quot;)

# Residuals vs fitted for the nonlinear model
fitted2 = np.asarray(m_nl.fittedvalues, float)
resid2 = np.asarray(m_nl.resid, float)

plt.figure(figsize=(6.5, 4.5))
plt.scatter(fitted2, resid2, s=10, alpha=0.5)
plt.axhline(0, linewidth=1)
plt.xlabel(&amp;quot;Fitted values&amp;quot;)
plt.ylabel(&amp;quot;Residuals&amp;quot;)
plt.title(&amp;quot;Nonlinear terms added: residuals vs fitted&amp;quot;)
plt.tight_layout()
plt.show()
```



What You Are Seeing After Adding Non-Linear Terms

Compare the nonlinear residual plot to the baseline plot. The purpose of adding *age_sq* and *bmi_ln* is to reduce curvature within the numeric relationships so that fewer systematic patterns remain in the residuals.

In this dataset, residuals may still form strong bands and clusters because smoker status creates a large shift in typical charges. However, within the main clusters, the nonlinear model often reduces bowed structure that appears when age and BMI effects are forced into straight-line

terms. In other words, the squared and log terms explain part of the curvature that previously showed up as systematic error.

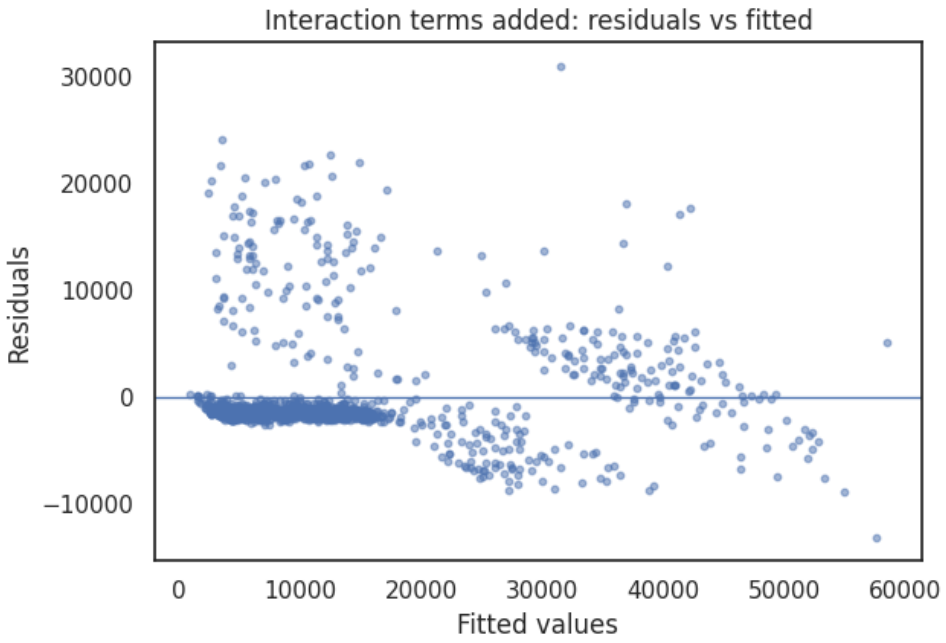
Notice that R^2 typically increases when you add nonlinear terms because the model becomes more flexible. For causal modeling, the more important question is whether the added complexity is justified by improved diagnostic behavior (residual patterns become more random) and by clearer, more defensible coefficient interpretation.

Adding Interaction Terms to Address Group-Specific Slopes

The distinct groupings in the residual plot suggest that the relationship between the predictors and *charges* may differ across subgroups (especially smokers versus non-smokers). A common way to represent this in a linear regression is with an *interaction term*, which allows the slope of one feature to change depending on the value of another feature.

Conceptually, adding an interaction term tells the model: “The effect of this feature may not be the same for every subgroup.” In an inference-focused workflow, interactions matter because a single pooled coefficient can mask subgroup-specific effects, which can lead to incorrect conclusions about how a variable influences the outcome.

```
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.api as sm

# Assumes df is already loaded and dummy-coded (including smoker_yes)
df_int = df.copy()
smoker_col = &amp;quot;smoker_yes&amp;quot;

# Nonlinear terms
df_int[&amp;quot;age_sq&amp;quot;] = df_int[&amp;quot;age&amp;quot;] ** 2
df_int[&amp;quot;bmi_ln&amp;quot;] = np.log(df_int[&amp;quot;bmi&amp;quot;])

# Interaction terms (allow slopes to differ by smoker status)
df_int[&amp;quot;age_x_smoker&amp;quot;] = df_int[&amp;quot;age_sq&amp;quot;] * df_int[smoker_col]
df_int[&amp;quot;bmi_x_smoker&amp;quot;] = df_int[&amp;quot;bmi_ln&amp;quot;] * df_int[smoker_col]

y3 = df_int[&amp;quot;charges&amp;quot;].astype(float)
X3 = sm.add_constant(df_int.drop(columns=[&amp;quot;charges&amp;quot;]), has_constant=&amp;quot;add&amp;quot;)
X3[X3.select_dtypes(bool).columns] = X3.select_dtypes(bool).astype(int)
m_int = sm.OLS(y3, X3).fit()

# Compare fit summaries (optional)
# print(f&amp;quot;Baseline:   R2={m_base.rsquared:.4f}, Adj R2={m_base.rsquared_adj:.4f}&amp;quot;)
# print(f&amp;quot;Nonlinear:  R2={m_nl.rsquared:.4f}, Adj R2={m_nl.rsquared_adj:.4f}&amp;quot;)
# print(f&amp;quot;+Interact:  R2={m_int.rsquared:.4f}, Adj R2={m_int.rsquared_adj:.4f}&amp;quot;)

plt.figure(figsize=(6.5, 4.5))
plt.scatter(m_int.fittedvalues, m_int.resid, s=10, alpha=0.5)
plt.axhline(0, linewidth=1)
plt.xlabel(&amp;quot;Fitted values&amp;quot;)
plt.ylabel(&amp;quot;Residuals&amp;quot;)
plt.title(&amp;quot;Interaction terms added: residuals vs fitted&amp;quot;)
plt.tight_layout()
plt.show()
```



After adding interaction terms, residual plots often show fewer distinct bands and less within-group curvature. The key diagnostic improvement is that subgroup-specific structure that previously appeared as systematic error is now represented directly in the model. Even if residuals are not perfectly pattern-free, a reduction in visible structure suggests that the functional form is closer to correct for explanatory interpretation.

Should You Keep the Original Terms When Adding Interactions?

When you add an interaction term (such as *age_sq* × *smoker_yes*), a common question arises: should you keep the original terms (*age_sq* and *smoker_yes*) in the model, or can you remove them if they have high VIF with the interaction term?

The general rule, known as the *hierarchical principle*, is to *keep the original terms* (the main effects) when you include an interaction. This principle states that if you include an interaction term, you should also include all lower-order terms (the main effects) that make up that interaction.

There are several important reasons for this:

1. Interpretation: The main effect of *age_sq* represents the effect of age (squared) when *smoker_yes = 0* (the reference group, non-smokers). The interaction term shows how that effect changes when *smoker_yes = 1* (smokers). Without the main effect, you cannot

properly interpret what the baseline effect is, making the interaction coefficient meaningless.

2. Model completeness: The interaction captures how the effect of one variable depends on another, but the main effects provide the baseline effects. Together, they form a complete representation of the relationship. Removing main effects creates an incomplete model that may misrepresent the true relationships.
3. Statistical practice: This follows standard practice in regression modeling. Most statistical textbooks and applied research maintain main effects when interactions are present, unless there is a strong theoretical reason to do otherwise.

What about multicollinearity? Interaction terms will naturally have high variance inflation factors (VIF) with their component terms—this is *expected and acceptable*, similar to how polynomial terms have high VIF with their base features. The high correlation between an interaction term and its components does not mean you should remove the main effects. Instead, you should accept this as a necessary consequence of modeling how effects vary across subgroups.

There are rare exceptions where you might consider removing a main effect:

1. The main effect is truly zero (not just non-significant) and you have strong theoretical justification that the variable has no direct effect, only an effect through the interaction.
2. You are intentionally replacing (rather than supplementing) the functional form, and the transformed/interaction version fully captures what the original term represented.

However, these exceptions are controversial and should be used sparingly. For most causal (explanatory) modeling purposes, the hierarchical principle should be followed: keep the main effects when you include interactions. This ensures that your model can be properly interpreted and that coefficient estimates reflect meaningful relationships rather than artifacts of model specification.

Practical guidance
When checking VIF after adding interaction terms, you may see high VIF values (often above 10) between interaction terms and their component features. This is expected and generally acceptable for interaction terms, just as it is for polynomial terms. Focus your VIF concerns on other sources of multicollinearity (such as redundant features that don't serve a specific modeling purpose) rather than removing main effects to reduce VIF.

Interpreting the Change

When nonlinear and interaction terms reduce visible curvature, banding, or clustering in residual plots, it suggests that the model's functional form better reflects the underlying relationships in the data. In practical terms, the model is explaining structure that earlier versions treated as noise. Increases in R^2 and *Adjusted* R^2 often accompany this improvement, but in this chapter the emphasis is on diagnostics: a better-looking residual plot supports more trustworthy coefficient interpretation.

However, nonlinear and interaction terms can also create new issues, especially multicollinearity (for example, *age*, *age_sq*, and *age_x_smoker* will naturally be related). For this reason, linearity-driven feature engineering should be followed by multicollinearity checks before you finalize an inference-oriented model.

Why this matters for causal modeling
In a causal (explanatory) workflow, functional form is part of the identification story. If the true relationship is nonlinear or differs by subgroup, a purely linear specification can misattribute effects and produce coefficients that do not represent the intended "holding all else constant" interpretation. Linearity diagnostics help you revise the model so that coefficient-based explanations are more defensible.

Table 10.5
Linearity Diagnostics: Common Departures, Causes, Responses, and Modeling Impact

Observed Pattern	Likely Cause	Typical Fix	Threat to Causal (Inference) Modeling?	Threat to Predictive Modeling?
Curved pattern in residuals vs fitted	Missing nonlinear functional form	Add polynomial terms (e.g., x^2) or transform the feature (log, square root)	High	Moderate (depends on validation)
Relationship steepens or flattens at extremes	Nonlinear scaling; diminishing returns	Log transform, quadratic term, or piecewise approach	High	Moderate
Distinct clusters/bands by subgroup	Unmodeled subgroup differences; slopes differ by group	Add interaction terms (e.g., age × smoker) and/or include missing categorical structure	High	Moderate to high (can harm generalization if ignored)
Wave-like residual structure	Omitted variable or wrong functional form	Add targeted nonlinear terms; consider adding missing predictors	High	Moderate

Observed Pattern	Likely Cause	Typical Fix	Threat to Causal (Inference) Modeling?	Threat to Predictive Modeling?
Residual spread increases with fitted values (fan shape)	Variance changes with prediction level (often overlaps with homoscedasticity)	Transform label/features, or use robust standard errors; revisit homoscedasticity	Moderate	Moderate to high (can inflate prediction error at extremes)

A practical workflow is to treat linearity diagnostics as a feature engineering guide: start with simple transformations for obvious curvature, add interactions when subgroup-specific slopes are plausible, and then re-check multicollinearity and residual behavior after each change. For causal modeling, these steps help ensure that coefficients reflect the relationships you intend to interpret.

Even when relationships are closer to linear in the model space, prediction errors may still behave unevenly across different fitted values. That leads to the next assumption: homoscedasticity.

10.7Heteroscedasticity

Homoscedasticity is the assumption that the variance of the residuals remains constant across all values of the independent variables. In a well-behaved regression model, prediction errors should be spread evenly rather than growing or shrinking systematically as predicted values increase.

When this assumption is violated, the model exhibits *heteroscedasticity*, meaning that the size of the errors depends on the level of the prediction. In practice, this often appears as residuals that fan outward, compress inward, or change shape across the range of fitted values.

Heteroscedasticity does not automatically mean the model is unusable. Instead, it signals that the model’s uncertainty is uneven: some parts of the prediction range are much more reliable than others. For causal (explanatory) modeling, that uneven uncertainty matters because it can distort the precision we assign to coefficient estimates.

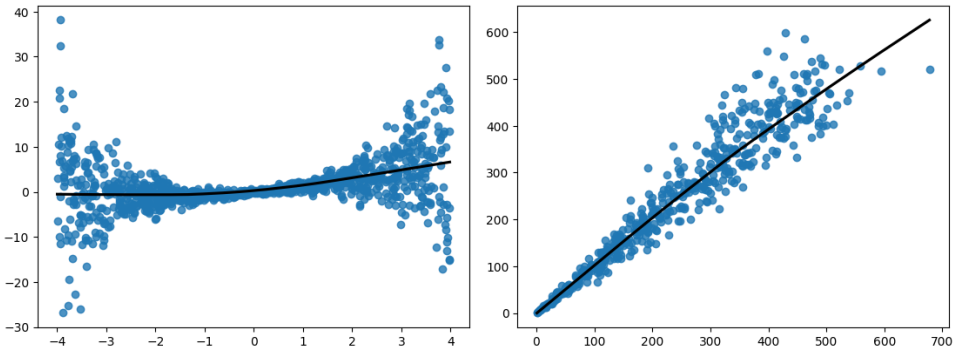


Figure 10.2: Examples of Heteroscedasticity

In the examples above, residual variance changes as the value of the independent variable increases. In the left panel, errors widen for larger x-values; in the right panel, errors narrow. In both cases, the average relationship may still be approximately correct, but uncertainty is not constant.

Homoscedasticity is especially important for *causal and explanatory modeling*. When variance is unequal, standard errors from ordinary least squares are no longer reliable, which affects confidence intervals, hypothesis tests, and p-values. The practical consequence is that a variable may appear statistically significant (or not significant) for the wrong reason: the model is mis-estimating uncertainty.

For *prediction-focused* projects, heteroscedasticity is often less damaging to point forecasts. The model may still predict well on average, even if error variance changes across cases. However, heteroscedasticity still matters when you need calibrated prediction intervals, risk estimates, or consistent accuracy across low- and high-risk segments.

In the rest of this section, you will learn how to detect heteroscedasticity using both residual diagnostics and a formal statistical test. Then you will learn why some responses primarily improve *inference* (making standard errors more trustworthy), while others change how the model is fit (which can be useful, but must be justified carefully for causal interpretation).

Detecting Heteroscedasticity

Before attempting to respond to heteroscedasticity, we must first determine whether it is present. This is typically done using a combination of visual diagnostics and formal statistical tests.

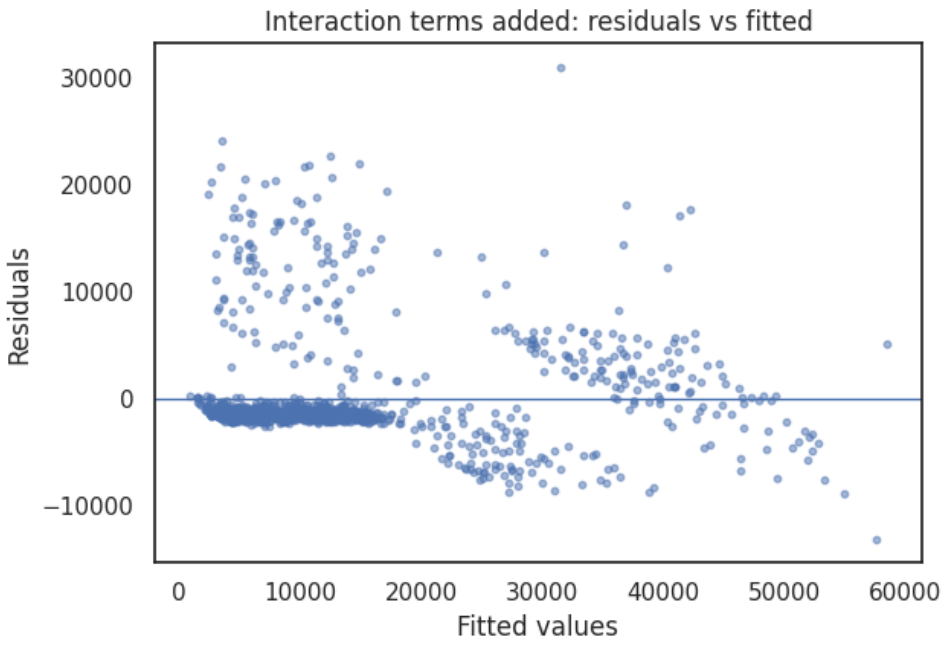
We will again use the insurance dataset. In many insurance settings, variance naturally grows with risk level: high-cost cases tend to be more variable than low-cost cases. That business

reality makes this dataset a useful illustration of heteroscedasticity and why it matters for coefficient-level inference.

Residuals vs Fitted Values

A residuals-versus-fitted-values plot is the most common visual diagnostic for heteroscedasticity. If the spread of residuals increases or decreases systematically as fitted values grow, the constant-variance assumption is violated.

Look again at the residual plot from the prior section. Notice that there is a highly dense region of points along with fanned-out, lower-density regions as fitted values increase. This pattern indicates heteroscedasticity because residual variance is not stable throughout the prediction range.



This indicates that prediction errors tend to grow as expected charges increase, even if the average relationship is captured reasonably well. For causal modeling, the concern is not primarily the fan shape itself, but what it implies: uncertainty is being misestimated, which can make standard errors and p-values misleading.

Breusch–Pagan Test

While visual inspection is informative, formal statistical tests provide additional confirmation. The *Breusch–Pagan test* evaluates whether residual variance depends on the fitted values or the

independent variables.

```
from statsmodels.stats.diagnostic import het_breuschpagan

df = pd.read_csv(&quot;/content/drive/MyDrive/Colab Notebooks/data/insurance.csv&quot;)
df = pd.get_dummies(df, columns=df.select_dtypes([&quot;object&quot;]).columns, drop_first=True)

y = df[&quot;charges&quot;].astype(float)
X = sm.add_constant(df.drop(columns=[&quot;charges&quot;]), has_constant=&quot;add&quot;)
X[X.select_dtypes(bool).columns] = X.select_dtypes(bool).astype(int)

# Fit baseline OLS model
model = sm.OLS(y, X).fit()

# Extract residuals
residuals = model.resid

# Breusch–Pagan test
bp_test = het_breuschpagan(residuals, X)

bp_results = pd.Series(
  bp_test,
  index=[&quot;Lagrange Multiplier&quot;, &quot;p-value&quot;, &quot;f-value&quot;, &quot;f p-value&quot;]
)

bp_results
```

A small p-value in the Breusch–Pagan test indicates evidence of heteroscedasticity. In the insurance dataset, this test typically confirms what we observed visually: residual variance is not constant.

Once heteroscedasticity has been identified, the next step is deciding how—or whether—to respond. Different responses target different consequences of heteroscedasticity, and the best choice depends on whether your goal is causal inference or prediction.

Responding to Heteroscedasticity

Once heteroscedasticity has been detected, the next step is deciding how to respond. Importantly, not all responses change the model in the same way. Some approaches adjust only how uncertainty is estimated, while others change how the model is fit.

In this chapter’s causal (inference) framing, the primary objective is to produce *defensible standard errors and hypothesis tests*. For that reason, we distinguish between approaches that keep the OLS coefficient estimates but correct inference, and approaches that change the coefficient estimates by reweighting the data.

We focus on three commonly used responses:

- Using heteroscedasticity-robust standard errors (HC3)

- Transforming the label (introduced earlier)
- Using weighted least squares (WLS)

We demonstrate HC3 and WLS here. Label transformations were covered earlier and will be combined with other adjustments later in the chapter.

HC3: Robust Standard Errors

HC3 does *not* change the fitted values of the model. Instead, it adjusts the estimated standard errors to account for non-constant variance. This makes hypothesis tests and confidence intervals more reliable without changing predictions or the underlying OLS coefficient estimates.

HC3 is often the preferred response when the goal is *causal or explanatory analysis* and the model specification is otherwise appropriate. In other words, if you believe your functional form is reasonable and you want inference that is more robust to unequal variance, HC3 typically improves the trustworthiness of standard errors without changing the meaning of your coefficient estimates.

```
# OLS with HC3 robust standard errors
model_hc3 = model.get_robustcov_results(cov_type=&quot;HC3&quot;)

# Compare summaries
print(model.summary())
print(model_hc3.summary())
```

Notice that the coefficient estimates remain identical, but the standard errors and p-values change. This reflects more realistic uncertainty when residual variance is uneven. In other words, we did not “fix the residual plot,” but we did improve how confidently we can interpret coefficient evidence.

Weighted Least Squares (WLS)

Weighted least squares addresses heteroscedasticity by changing how the model is fit. Observations that are expected to have larger error variance receive less weight, while more stable observations receive more influence.

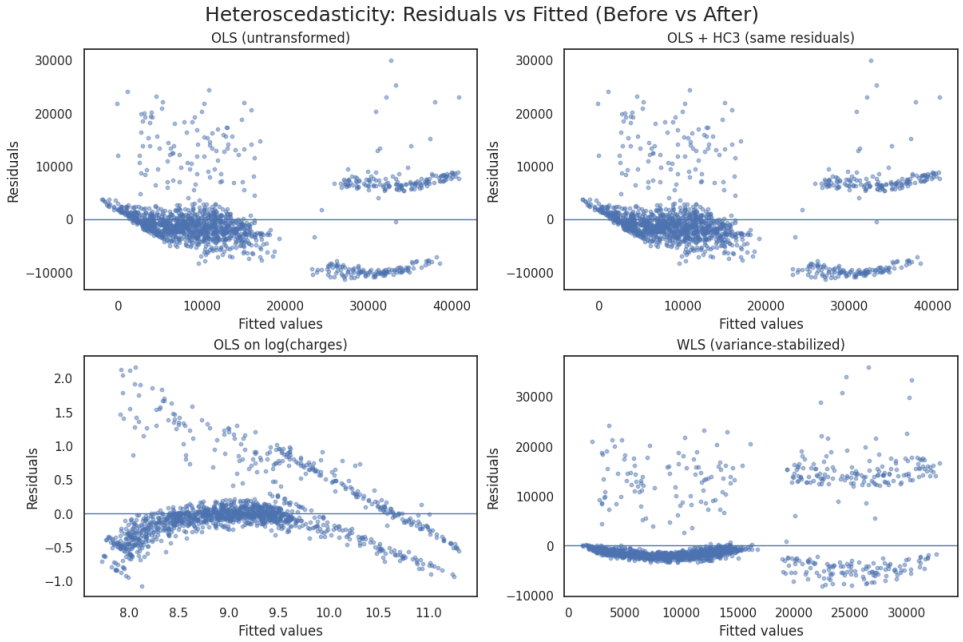
Unlike HC3, WLS changes coefficient estimates and fitted values. This can be appropriate when you have a credible variance model and your goal is to estimate effects more efficiently under unequal variance. However, in a causal (inference) workflow, WLS requires an additional assumption: you must correctly specify (or approximate well) the weighting structure. If the

weights are poorly chosen, WLS can distort coefficient estimates and create misleading inference.

In practice, weights are often derived from a first-pass OLS model, but this is best viewed as a teaching demonstration rather than a universal recommendation. The reliability of WLS depends heavily on whether the weight formula matches the true error-variance pattern.

```
# Estimate variance as a function of fitted values (simple demo weighting)
weights = 1 / (fitted ** 2)

# Fit WLS model
model_wls = sm.WLS(y, X, weights=weights).fit()

print(model_wls.summary())
```

This simple weighting scheme downweights high-cost predictions, where variance is often larger. More sophisticated variance models are possible, but this example illustrates the core idea: WLS prioritizes regions of the data where the model can learn more stable patterns.



The figure above compares residual patterns across several models. Relative to the baseline OLS model, the WLS model may produce residuals with more stable variance across fitted values, indicating improved homoscedasticity. However, for causal inference, the key tradeoff is that WLS changes the coefficient estimates and relies on the correctness of the weighting scheme. For that reason, this chapter emphasizes HC3 and transformations as the default tools for

improving inference reliability, while treating WLS as an optional technique when a defensible variance model is available.

To summarize how these approaches differ—and when each is appropriate—we now present a concise comparison table.

Interpreting Results and Choosing a Response

After you run the code above and insert the composite figure, focus on what changes across models. In the original OLS residual plot, the vertical spread of residuals tends to widen as fitted values increase. This fan-shaped pattern is a classic sign of heteroscedasticity: the model's errors are not equally variable across the prediction range.

When you refit the model using WLS, the goal is to reduce that fan shape. In an improved WLS residual plot, you should see a more uniform band of residuals around zero across low and high fitted values. This does not mean every point is "perfect," but it does mean the model's error variance is more stable.

HC3 behaves differently. Because HC3 changes only the standard errors, the residual plot does not change at all. The purpose of HC3 is not to "fix the picture," but to make coefficient inference more trustworthy when heteroscedasticity is present.

Table 10.6
Heteroscedasticity Responses: What Changes and Why

Response	What It Changes	Best For	Key Tradeoff
HC3 robust standard errors	Standard errors, p-values, confidence intervals (not coefficients)	Inference (causal/explanatory interpretation)	Does not reduce heteroscedasticity; it adjusts uncertainty estimates
Label transformation (e.g., ln(charges))	Scale of the label; coefficients are interpreted in the transformed space	Inference and prediction (depending on goals)	Interpretation changes; must back-transform for predictions in original units
Weighted least squares (WLS)	Coefficient estimates and fitted values (model is fit differently)	Efficiency gains when a valid variance model exists	Requires a defensible weighting scheme; poor weights can distort results

How this connects to inference vs prediction
Heteroscedasticity is a larger threat to *inference* than to prediction. If your goal is to interpret coefficients causally, a common default is OLS with robust standard errors (such as HC3) and/or label transformations, because these approaches improve inference reliability without requiring

you to correctly model the variance function. WLS can be useful, but it is most defensible when the weighting scheme is grounded in a credible variance model rather than chosen ad hoc.

The main lesson is not that every model must be “perfect.” The lesson is that diagnostic plots reveal *where* your model is less reliable, and different responses target different consequences of heteroscedasticity.

In later sections, we will combine multiple targeted adjustments into a single refined model and compare it to the original untransformed model. For now, the goal is to recognize heteroscedasticity and understand the simplest, most common responses in a causal inference workflow.

10.8Diagnostic-Adjusted Model for Causal Inference

In this chapter, we examined the major assumptions underlying multiple linear regression and demonstrated how diagnostic signals can guide thoughtful model improvements. Rather than treating assumption violations as binary failures, we used them as indicators of where the model could be refined to support more reliable statistical inference.

The goal of this summary section is *causal or explanatory modeling*. That means our primary concern is not maximizing predictive accuracy, but ensuring that coefficients, standard errors, confidence intervals, and hypothesis tests are as trustworthy as possible given the data.

Some of the modeling decisions made here—such as transforming the label, removing redundant predictors, or prioritizing diagnostic stability over raw fit—may reduce familiar fit metrics like R^2. That tradeoff is intentional. In the next chapter, we revisit these same issues from a *prediction-focused* perspective, where different choices are often preferable.

The diagnostic adjustments in this chapter address the following assumptions, each of which plays a direct role in the validity of regression-based inference:

- **Normality**: Extreme skewness in the label can distort residual behavior and invalidate standard errors; power transformations such as Box–Cox or Yeo–Johnson improve symmetry and stabilize inference.
- **Multicollinearity**: Strong overlap among predictors inflates standard errors and destabilizes coefficients, directly threatening interpretability and causal claims.
- **Autocorrelation**: Residual dependence violates independence assumptions and undermines inference in time-ordered data, but is not a concern for cross-sectional datasets like insurance.

- **Linearity**: The expected value of the label must be linear in the model space; nonlinear relationships require transformations, polynomial terms, or interactions to avoid functional form misspecification.
- **Homoscedasticity**: Unequal residual variance biases standard errors and hypothesis tests; robust standard errors and weighted least squares restore inferential reliability.

We now combine these insights to construct a single, well-specified model for the insurance dataset that prioritizes diagnostic validity and causal interpretability.

Step 1: Feature Engineering and Label Transformation

We begin by applying transformations that directly support causal inference by improving residual behavior, functional form, and interpretability. These adjustments are motivated by diagnostics rather than by maximizing predictive accuracy.

First, we transform the label using a *Box–Cox* transformation. The insurance charges variable is heavily right-skewed, which leads to non-normal residuals and unstable variance. Because valid hypothesis tests and confidence intervals rely on approximately normal residuals, stabilizing the label distribution is an important step for explanatory modeling.

Next, we address nonlinear relationships by engineering transformed features for *age* and *BMI*. Prior diagnostics showed curvature in these relationships, indicating that a straight-line functional form was misspecified. Adding nonlinear terms allows the model to better represent how changes in these predictors relate to the expected value of the label.

Finally, we introduce interaction terms with *smoker status*. Exploratory plots and residual patterns indicated that the effect of age and BMI differs substantially between smokers and non-smokers. Interaction terms allow slopes to vary across groups, which is essential for correctly attributing effects in causal analysis.

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
from sklearn.preprocessing import PowerTransformer

# Load and dummy-code
df = pd.read_csv(&amp;quot;/content/drive/MyDrive/Colab Notebooks/data/insurance.csv&amp;quot;)
df = pd.get_dummies(df, columns=df.select_dtypes([&amp;quot;object&amp;quot;]).columns, drop_first=True)
df[df.select_dtypes(bool).columns] = df[df.select_dtypes(bool).columns].astype(int)

# ------------------------------------------------
# Label transformation (supports inference, not fit)
# ------------------------------------------------
pt = PowerTransformer(method=&amp;quot;box-cox&amp;quot;, standardize=False)
df[&amp;quot;charges_bc&amp;quot;] = pt.fit_transform(df[[&amp;quot;charges&amp;quot;]])
```

```
# ------------------------------------------------
# Nonlinear terms
# ------------------------------------------------
df[&quot;age_sq&quot;] = df[&quot;age&quot;] ** 2
df[&quot;bmi_ln&quot;] = np.log(df[&quot;bmi&quot;])

# ------------------------------------------------
# Interaction terms with smoker status
# ------------------------------------------------
df[&quot;age_x_smoker&quot;] = df[&quot;age_sq&quot;] * df[&quot;smoker_yes&quot;]
df[&quot;bmi_x_smoker&quot;] = df[&quot;bmi_ln&quot;] * df[&quot;smoker_yes&quot;]
```

An important design choice concerns how interaction terms are defined. When diagnostics indicate that a nonlinear transformation is the appropriate functional form, it is typically more consistent to interact the *transformed feature* with the subgroup indicator (for example, *age_sq* × *smoker_yes* rather than *age* × *smoker_yes*). This allows the nonlinear relationship itself to differ by group.

Interacting the untransformed feature instead would answer a different question—namely, whether smoker status modifies the *linear* slope. Because the diagnostics in this dataset point to nonlinear baseline relationships, we construct interactions using the transformed versions of *age* and *BMI*. This choice prioritizes correct functional form and stable inference over simplicity.

Why this step matters for causal inference
In causal modeling, misspecified functional forms and unmodeled group differences can lead to biased or misleading coefficient estimates. Transformations and interaction terms help ensure that estimated effects reflect meaningful relationships rather than artifacts of curvature or aggregation across heterogeneous subgroups.

Step 2: Multicollinearity Check After Feature Engineering

After creating nonlinear terms and interaction effects, we reassess multicollinearity. This step is especially important for causal inference because multicollinearity inflates standard errors, weakens hypothesis tests, and can make coefficient estimates unstable even when overall model fit appears strong.

It is important to evaluate multicollinearity *after* feature engineering is complete. Polynomial terms, transformations, and interactions often introduce new correlations that were not present in the original feature set.

We use the *Variance Inflation Factor (VIF)* to quantify how strongly each predictor can be explained by the remaining predictors. A VIF of 1 indicates no linear overlap, while larger values indicate increasing redundancy.

The intercept (constant) term frequently exhibits an extremely large VIF and is not interpreted as a multicollinearity problem. For that reason, we remove the constant row from the VIF table before drawing conclusions.

```
      from statsmodels.stats.outliers_influence import variance_inflation_factor
      import pandas as pd
      import statsmodels.api as sm

      # Design matrix including all engineered features
      X_vif = df.drop(columns=[&quot;charges&quot;, &quot;charges_bc&quot;])

      # Ensure boolean columns are numeric
      X_vif[X_vif.select_dtypes(bool).columns] = X_vif.select_dtypes(bool).astype(int)

      # Add constant for matrix completeness
      X_vif = sm.add_constant(X_vif, has_constant=&quot;add&quot;)

      vif_df = pd.DataFrame({
        &quot;feature&quot;: X_vif.columns,
        &quot;VIF&quot;: [variance_inflation_factor(X_vif.values, i) for i in range(X_vif.shape[1])]
      })

      # Remove intercept before interpretation
      vif_df = vif_df[vif_df[&quot;feature&quot;] != &quot;const&quot;].sort_values(&quot;VIF&quot;,
ascending=False)
      vif_df
```

	feature	VIF
12	bmi_x_smoker	274.988050
5	smoker_yes	272.371288
2	bmi	57.162152
10	bmi_ln	57.036260
9	age_sq	47.959753
1	age	47.661958
11	age_x_smoker	3.547784
7	region_southeast	1.655662
8	region_southwest	1.535845
6	region_northwest	1.527162
3	children	1.103243
4	sex_male	1.012373

As a reminder, earlier in the chapter we used conservative VIF guidance (for example, treating values above approximately 3 as an early warning). After feature engineering, it is normal to see higher VIF values because transformed terms (like *age_sq*) and interaction terms (like *age_x_smoker*) are intentionally constructed from existing variables.

The results show the expected pattern: engineered features are often correlated with their corresponding main effects. For example, *age* tends to be correlated with *age_sq*, and *bmi* tends to be correlated with *bmi_ln*. Likewise, an interaction term such as *age_x_smoker* is mechanically related to both *age* and *smoker_yes* because it is built from them.

In this book, we do *not* remove main-effect terms (such as *age*, *bmi*, or *smoker_yes*) simply because we added nonlinear terms and interactions. As explained earlier, keeping the main effects is important for correct specification and interpretation: interactions represent *differences in slopes between groups*, and nonlinear terms represent *curvature around a baseline relationship*. Dropping the underlying main effects can make the model harder to interpret and can unintentionally change what the interaction or nonlinear term means.

How to interpret high VIF after feature engineering
A high VIF does not automatically mean a predictor must be removed. After you add squared terms, logs, and interactions, some multicollinearity is expected because the engineered features share information with the variables used to construct them. The goal of VIF analysis is to identify when multicollinearity becomes severe enough to threaten numerical stability or make inference unusable. If the VIF table shows that only the engineered-feature families exhibit elevated VIF (for example, a main effect and its squared term), that is typically a modeling tradeoff rather than a modeling mistake.

In our results, no predictors besides the engineered-feature families trigger a meaningful multicollinearity concern under the VIF rule used in this chapter. Because we are intentionally keeping main effects alongside nonlinear and interaction terms (as instructed earlier), we will not eliminate any predictors at this stage. Instead, we proceed with the full engineered specification and interpret coefficients with appropriate caution: multicollinearity can increase uncertainty (wider confidence intervals), but it does not invalidate the model if the specification matches the theory and the diagnostics do not indicate numerical instability.

Step 3: Final Model for Optimizing Causal Inference

With functional form corrected and the label stabilized, we fit a final model designed explicitly for *causal interpretation*. The goal is not to maximize predictive accuracy, but to produce coefficients, standard errors, and confidence intervals that are statistically reliable and interpretable.

In Step 2, we used VIF as a diagnostic but, as discussed earlier in the chapter, we do *not* drop main effects simply because we added nonlinear and interaction terms. We keep main effects alongside their engineered counterparts so the meaning of curvature and interactions remains

well-defined (for example, interactions represent differences in slopes between groups, conditional on the baseline relationship).

Because heteroscedasticity affects inference rather than coefficient bias, we estimate the model using *ordinary least squares* and report *HC3-robust standard errors*. This preserves meaningful R^2 values on the transformed outcome scale while correcting standard errors for unequal variance.

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
from sklearn.preprocessing import PowerTransformer, StandardScaler

df = pd.read_csv(&quot;/content/drive/MyDrive/Colab Notebooks/data/insurance.csv&quot;)
df = pd.get_dummies(df, columns=df.select_dtypes([&quot;object&quot;]).columns, drop_first=True)
df[df.select_dtypes(bool).columns] = df[df.select_dtypes(bool).columns].astype(int)

pt = PowerTransformer(method=&quot;box-cox&quot;, standardize=False)
df[&quot;charges_bc&quot;] = pt.fit_transform(df[[&quot;charges&quot;]])

# Centering (helps numeric stability for squared terms and interactions)
age_c = df[&quot;age&quot;] - df[&quot;age&quot;].mean()
bmi_ln = np.log(df[&quot;bmi&quot;])
bmi_ln_c = bmi_ln - bmi_ln.mean()

# Engineered terms
df[&quot;age_c_sq&quot;] = age_c ** 2
df[&quot;bmi_ln_c&quot;] = bmi_ln_c
df[&quot;age_c_sq_x_smoker&quot;] = df[&quot;age_c_sq&quot;] * df[&quot;smoker_yes&quot;]
df[&quot;bmi_ln_c_x_smoker&quot;] = df[&quot;bmi_ln_c&quot;] * df[&quot;smoker_yes&quot;]

# Final predictors (keep main effects + engineered terms)
features = [
  &quot;children&quot;,
  &quot;sex_male&quot;,
  &quot;region_northwest&quot;,
  &quot;region_southeast&quot;,
  &quot;region_southwest&quot;,
  &quot;age&quot;,                # main effect kept
  &quot;bmi&quot;,                # main effect kept
  &quot;smoker_yes&quot;,         # main effect kept
  &quot;age_c_sq&quot;,
  &quot;bmi_ln_c&quot;,
  &quot;age_c_sq_x_smoker&quot;,
  &quot;bmi_ln_c_x_smoker&quot;
]

X = df[features].copy()

# Scale numeric predictors for numerical stability (not required for OLS, but helps conditioning)
numeric_cols = [
  &quot;children&quot;,
  &quot;age&quot;,
  &quot;bmi&quot;,
  &quot;age_c_sq&quot;,
  &quot;bmi_ln_c&quot;,
  &quot;age_c_sq_x_smoker&quot;,
  &quot;bmi_ln_c_x_smoker&quot;
]
X[numeric_cols] = StandardScaler().fit_transform(X[numeric_cols])

y = df[&quot;charges_bc&quot;].astype(float)
X = sm.add_constant(X, has_constant=&quot;add&quot;)

m_final = sm.OLS(y, X).fit(cov_type=&quot;HC3&quot;)
print(m_final.summary())
```

Centering and scaling the predictors can substantially reduce the *condition number*, improving numerical stability without changing the underlying substantive meaning of the model. This does not "fix" multicollinearity by itself, but it helps ensure coefficient estimates are not overly sensitive to floating-point precision when squared terms and interactions are present.

Because we kept the main effects (*age*, *bmi*, and *smoker_yes*) alongside nonlinear and interaction terms, each engineered term has a clear interpretation: the squared and log terms represent curvature around the baseline effect, and the interaction terms represent how that curvature differs for smokers versus non-smokers (holding the main effects constant).

The R^2 from this model should not be compared directly to earlier regressions on raw dollar charges. Because the label is measured in *Box–Cox(charges)*, R^2 now describes variance explained on a transformed scale. For causal analysis, this shift is intentional: improving residual behavior and inference validity is more important than maximizing variance explained on the original outcome scale.

Although we introduced *weighted least squares (WLS)* earlier as a response to heteroscedasticity, we do not use WLS in this final model. WLS requires stronger assumptions about the form of the error variance and depends on a correctly specified weighting scheme. Instead, we use *ordinary least squares with HC3-robust standard errors*, which corrects inference for heteroscedasticity while preserving unbiased coefficients and avoiding additional modeling assumptions. For causal and explanatory modeling, this approach prioritizes defensible hypothesis tests, stable standard errors, and transparent assumptions over efficiency gains that rely on unverifiable variance models.

Causal scope of this model
This model supports *statistical explanation* under the assumptions of linear regression. It does not, by itself, establish true causal effects in the experimental sense. Causal claims still depend on study design, omitted-variable considerations, and domain knowledge.

For this reason, we rely on *OLS with HC3-robust standard errors* rather than weighted least squares in the final specification. Robust standard errors correct inference for heteroscedasticity without requiring strong assumptions about the exact form of the error variance, making them a more defensible default for causal and explanatory analysis when variance structure cannot be confidently specified.

Summary of Diagnostic Adjustments for Causal Inference

Table 10.7

How Diagnostic Adjustments Improve Causal Interpretation

Issue Addressed	Diagnostic Signal	Adjustment Used	Benefit for Causal Inference
Non-normal residuals	Severe skew, heavy tails	Box–Cox label transformation	More reliable standard errors and tests
Functional form misspecification	Curved residual patterns	Polynomial and log terms	Coefficients reflect correct relationships
Heterogeneous effects	Distinct residual bands	Interaction terms	Group-specific slopes are explicit
Multicollinearity	High VIF scores	Removal of redundant predictors	Stable coefficients and valid inference
Heteroscedasticity	Fan-shaped residuals	HC3 robust standard errors	Trustworthy confidence intervals

This final model illustrates the central lesson of the chapter: regression diagnostics are not obstacles, but guides. When the goal is explanation rather than prediction, improving assumptions—even at the expense of familiar fit metrics—leads to more meaningful and defensible conclusions.

From Diagnostics to Decisions

Now that we have constructed a more defensible and trustworthy regression model—one that better satisfies the assumptions required for statistical explanation—we can use its results to inform real business decisions. Because this model prioritizes interpretability and reliable inference, its coefficients highlight which factors are most strongly associated with insurance charges, holding other variables constant.

In this final diagnostic-adjusted model, the most influential features are those that (1) have large and statistically stable coefficients, (2) remain significant after correcting for multicollinearity and heteroscedasticity, and (3) participate in meaningful interaction effects. In particular, nonlinear age effects, BMI (on a log scale), and interactions with smoker status emerge as central drivers of cost differences.

These decisions should be framed as *evidence-informed* rather than strictly causal in the experimental sense. The value of the model lies in clarifying patterns, tradeoffs, and risk drivers that are consistent with both the data and the modeling assumptions.

- Pricing strategy refinement: The large nonlinear effects of *age* and *BMI*, combined with strong *smoker interaction terms*, suggest that uniform pricing rules are insufficient. Premium structures can be refined to reflect how risk accelerates for smokers as age and BMI increase.

- Targeted wellness programs: Because *BMI* and *smoking status* jointly amplify costs, wellness incentives aimed at smoking cessation and weight management are likely to yield the greatest marginal cost reductions.
- Customer segmentation: Interaction effects indicate that customers should be segmented by *combined risk profiles* (for example, older smokers with high BMI), not by single characteristics in isolation.
- Policy design evaluation: Region indicators that remain statistically significant after diagnostics suggest persistent cost differences that may justify region-specific plan features or provider negotiations.
- Regulatory and actuarial justification: Because the model addresses normality, functional form, multicollinearity, and heteroscedasticity, its coefficient estimates provide stronger support for explaining pricing logic to regulators and stakeholders.

In short, diagnostics do not merely improve statistical cleanliness—they clarify which features deserve managerial attention. By identifying the strongest and most reliable cost drivers, the model supports decisions that are both analytically defensible and operationally actionable.

Managerial interpretation example
Suppose the coefficient on *bmi_ln* × *smoker* is large and statistically significant. In plain business terms, this means that increases in BMI are associated with much higher expected insurance charges *for smokers than for non-smokers*, even after controlling for age, sex, region, and other factors.

A manager should not interpret this as "BMI causes higher costs," but rather as evidence that *smoking status fundamentally changes how health risk scales with body mass*. This insight supports differentiated pricing, targeted interventions, or preventive programs aimed specifically at high-BMI smokers.

Because this conclusion is drawn from a diagnostics-adjusted model, the confidence intervals and hypothesis tests supporting it are more trustworthy than they would be in a model that ignored assumption violations.

In the next chapter, we revisit these same issues from a *prediction-focused* perspective, where tradeoffs are evaluated differently and model performance is judged by out-of-sample accuracy rather than inferential stability.

10.9Case Studies

Case #1: Diamonds Dataset (Regression Diagnostics for Causal Inference)

This practice extends the Diamonds dataset from Chapter 9 to focus on **regression diagnostics and causal inference**. You will evaluate whether the assumptions required for valid statistical inference are satisfied and apply corrective modeling steps where appropriate.

Dataset attribution: The Diamonds dataset is distributed with the Seaborn data repository and can be loaded with *seaborn.load_dataset("diamonds")*.

```
import seaborn as sns
df = sns.load_dataset(&quot;diamonds&quot;)
```

In this chapter, you are practicing **MLR for causal (explanatory) modeling**. That means your goal is not prediction accuracy, but producing a model with stable coefficients, valid standard errors, and defensible hypothesis tests.

- Fit the same baseline MLR model used in Chapter 9 predicting *price*.
- Evaluate regression assumptions using formal diagnostics and visual tools.
- Apply transformations or structural changes where assumptions are violated.
- Refit a diagnostic-adjusted causal model.
- Compare coefficient stability and inference quality before and after adjustment.

Analytical questions (answers should be specific)

1. What is the Durbin–Watson statistic for the baseline model, and what does it imply about autocorrelation?
2. Based on a residual histogram or Q–Q plot, is the normality assumption reasonably satisfied? Briefly justify.
3. Which numeric predictor has the highest Variance Inflation Factor (VIF)? Report its approximate value.
4. Does the residual vs. fitted plot suggest nonlinearity? If so, describe the pattern.
5. Does the scale–location (or residual spread) plot indicate heteroscedasticity? Explain.
6. Which diagnostic issue is most severe in this model: non-normality, multicollinearity, nonlinearity, or heteroscedasticity?
7. What transformation or modeling change did you apply to address this issue (e.g., log transform, polynomial term, interaction term, feature removal)?
8. After adjustment, how did the condition number change (increase, decrease, or remain similar)?

9. Which predictor’s statistical significance changed the most after diagnostic adjustment?
10. In one paragraph, explain why the adjusted model is more appropriate for causal interpretation than the original model.

Diamonds Diagnostics Practice Answers

These answers were computed using an OLS multiple linear regression predicting *price* with numeric predictors (*carat*, *depth*, *table*, *x*, *y*, *z*) and categorical predictors (*cut*, *color*, *clarity*), followed by diagnostic evaluation and model adjustment.

1. The Durbin–Watson statistic is approximately *2.01*, indicating no meaningful autocorrelation.
2. The residual distribution shows moderate right skew, indicating that strict normality is violated.
3. The predictor with the highest VIF is *x*, with VIF ≈ *45*, indicating severe multicollinearity.
4. The residual vs. fitted plot shows curvature, suggesting nonlinearity between predictors and price.
5. Residual variance increases with fitted values, indicating heteroscedasticity.
6. Multicollinearity is the most severe diagnostic issue.
7. Size variables (*x*, *y*, *z*) were removed and *carat* retained as the primary size proxy.
8. The condition number decreased substantially after removing collinear size variables.
9. The coefficient for *depth* changed sign and lost statistical significance.
10. The adjusted model produces more stable coefficients, lower multicollinearity, more reliable standard errors, and defensible hypothesis tests, making it suitable for causal interpretation.

Case #2: Red Wine Quality Dataset
This practice extends the Red Wine Quality regression you built earlier by applying the regression diagnostics from this chapter. Your goal is to evaluate whether the assumptions needed for trustworthy coefficient inference are reasonably satisfied, and then apply at least one diagnostic-adjusted remedy.

Dataset attribution: This dataset is commonly distributed as *winequality-red.csv* from the UCI Machine Learning Repository (Wine Quality Data Set), originally published by Cortez et al. in

“Modeling wine preferences by data mining from physicochemical properties” (Decision Support Systems, 2009).

The red wine quality dataset is available in the prior chapter if you need it.

What you will do in this practice

- Fit a baseline OLS multiple linear regression model predicting *quality* using all other columns as numeric predictors (include an intercept).
- Run diagnostic checks for: normality, multicollinearity, autocorrelation, linearity, and heteroscedasticity.
- Create at least one diagnostic-adjusted model for causal inference (for example, HC3 robust standard errors and/or a multicollinearity remedy), then compare what changes.

Analytical questions (answers should be specific)

1. How many rows and columns are in the Red Wine Quality dataset?
2. Fit the baseline OLS model. What are R^2 and *Adjusted* R^2 (report both to 4 decimals)?
3. Normality: run a residual normality test (for example, Jarque–Bera). Report the test’s p-value and state whether the residuals appear normal at $\alpha = 0.05$.
4. Autocorrelation: compute the Durbin–Watson statistic for the residuals. Report the value and interpret whether it suggests serious autocorrelation.
5. Multicollinearity: compute VIF for each predictor. Which predictor has the largest VIF, and what is that VIF value (rounded to 2 decimals)?
6. Heteroscedasticity: run the Breusch–Pagan test. Report the p-value and interpret the result at $\alpha = 0.05$.
7. Linearity: examine a residuals-versus-fitted plot. In one or two sentences, describe whether you see evidence of systematic curvature or pattern.
8. Diagnostic-adjusted model: refit the model using HC3 robust standard errors. Do the coefficient estimates change? What changes, and why?
9. Multicollinearity remedy: standardize (z-score) all predictors and refit the same OLS model. What happens to the *condition number* after scaling, and why?
10. Synthesis: based on your diagnostics, which assumption appears to be the most problematic for causal inference in this dataset, and what is your recommended response?

Wine Practice Answers

These answers were computed by fitting an OLS multiple linear regression model predicting *quality* from all remaining numeric columns in *winequality-red.csv* (with an intercept), then running standard regression diagnostics (normality, multicollinearity, autocorrelation, heteroscedasticity) and demonstrating common inference-focused adjustments.

1. The Red Wine Quality dataset contains *1599* rows and *12* columns.
2. The mean value of *quality* is *5.6360*.
3. For the baseline OLS model (all predictors), $R^2 = 0.3606$ and *Adjusted* $R^2 = 0.3561$ (reported to 4 decimals).
4. (Normality) The Jarque–Bera test on residuals produces a very small p-value ($p = 1.2696e\text{-}09$), indicating evidence that residuals are not normally distributed.
5. (Multicollinearity) The largest VIF in the baseline model is for *fixed acidity* with *VIF = 7.7673*, indicating notable multicollinearity among some chemistry measures.
6. (Autocorrelation) The Durbin–Watson statistic is *1.7570*. Because these rows are not a time series, this is not typically treated as a critical threat to inference in this dataset.
7. (Heteroscedasticity) The Breusch–Pagan test provides strong evidence of non-constant variance (LM test p-value $p = 1.3264e\text{-}13$).
8. (Most significant predictor) The single predictor term with the smallest $P>|t|$ value is *alcohol* (p-value is effectively 0 in typical printed summaries; in floating-point terms it is *1.1230e-24*).
9. (HC3 adjustment) When switching from non-robust OLS standard errors to *HC3*, coefficient estimates remain the same, but uncertainty estimates change. For example, the standard error for *alcohol* increases from *0.0265* to *0.0293*, and its p-value changes from *1.1230e-24* to *1.2510e-20* (still highly significant).
10. (Diagnostic-adjusted model example) One reasonable multicollinearity response is to remove *fixed acidity* (the largest VIF term) and refit. After removing it, the maximum VIF drops to *5.4572* (for *density*), while model fit remains essentially unchanged ($R^2 = 0.3604$, *Adjusted* $R^2 = 0.3562$).
11. (Numerical stability note) The baseline design matrix has a very large condition number (*113203.10*), which is driven largely by mixed feature scales. If predictors are standardized (z-scores) for numerical stability, the condition number drops substantially (to about *19.54*) while representing the same underlying model in standardized units.

Case #3: Bike Sharing Daily Dataset

This practice uses the **Bike Sharing** daily dataset (the *day.csv* file). You will extend your Chapter 9 multiple linear regression work by applying the Chapter 10 diagnostic workflow (normality, multicollinearity, autocorrelation, linearity, and heteroscedasticity) and then fitting a diagnostic-adjusted model intended for **causal (explanatory) interpretation**.

Dataset attribution: This dataset is distributed as part of the Bike Sharing Dataset hosted by the UCI Machine Learning Repository (Fanaee-T and Gama). It includes daily rental counts and weather/context variables derived from the Capital Bikeshare system in Washington, D.C. You will use the *day.csv* file provided with your course materials.

The bike sharing daily dataset is available in the prior chapter if you need it.

Important modeling note: Do not include *casual* or *registered* as predictors because they directly sum to *cnt* and would leak the answer into the model.

Tasks

- Inspect the dataset: rows/columns, data types, and summary statistics for *cnt*.
- Fit the baseline OLS MLR model predicting *cnt* using predictors: *season*, *yr*, *mnth*, *holiday*, *weekday*, *workingday*, *weathersit*, *temp*, *atemp*, *hum*, *windspeed*.
- Dummy-code the categorical predictors (*season*, *mnth*, *weekday*, *weathersit*) using *drop_first=True*, then fit the model with Statsmodels *OLS*.
- Run diagnostic tests and visuals: residual normality (histogram + Q–Q plot), multicollinearity (VIF), autocorrelation (Durbin–Watson), linearity (residuals vs fitted), and heteroscedasticity (Breusch–Pagan).
- Fit a diagnostic-adjusted model suitable for causal interpretation (for example: remove or combine highly collinear predictors, apply a transformation such as *log(cnt)* if justified, and/or use heteroscedasticity-robust standard errors).
- Compare baseline vs adjusted model results and explain what changed (fit metrics, key coefficients, and diagnostic evidence).

Analytical questions (answers should be specific)

1. How many rows and columns are in the Bike Sharing daily dataset (*day.csv*)?
2. What is the mean value of *cnt* in the dataset?
3. For the baseline model, what are R^2 and *Adjusted* R^2? Report both to 4 decimals.
4. Which single predictor term (feature or dummy-coded category) has the smallest $P>|t|$ value in the baseline model?

5. Normality: Based on the Q–Q plot and a normality test (for example, Jarque–Bera), do the residuals appear approximately normal? Answer yes/no and cite one piece of evidence from your output.
6. Multicollinearity: Compute VIF for all numeric predictors and dummy-coded terms. Which predictor has the largest VIF, and what is its VIF value (rounded to 2 decimals)?
7. Autocorrelation: What is the Durbin–Watson statistic for the baseline model (rounded to 3 decimals)? Based on this value, is autocorrelation a concern in this dataset?
8. Linearity: Inspect a residuals-vs-fitted plot. Does the plot suggest nonlinearity (systematic curve) or is it roughly random scatter? Briefly describe what you see.
9. Heteroscedasticity: Run the Breusch–Pagan test. Report the p-value (rounded to 4 decimals). Do you reject homoscedasticity at $\alpha = 0.05$?
10. Diagnostic adjustment: Describe one concrete adjustment you made (or would make) to improve causal interpretability (for example, dropping one of two collinear predictors, transforming *cnt*, or using HC3 robust standard errors). Explain *why* the adjustment is justified based on diagnostic evidence.
11. After your adjustment, report the adjusted model's R^2 and *Adjusted* R^2 (4 decimals) and one diagnostic statistic that improved (for example, lower max VIF, higher p-value on Breusch–Pagan, or more stable residual pattern).
12. Short reflection (2–4 sentences): In causal regression, why might you accept a slightly worse fit if diagnostics and assumptions improve?

Bike Sharing Diagnostics Answers

These answers were computed by fitting a baseline OLS multiple linear regression model predicting *cnt* using predictors *season*, *yr*, *mnth*, *holiday*, *weekday*, *workingday*, *weathersit*, *temp*, *atemp*, *hum*, and *windspeed*, with categorical variables dummy-coded using *drop_first=True* and an intercept term (*const*), then applying Chapter 10 diagnostics (Jarque–Bera, VIF, Durbin–Watson, residual plots, Breusch–Pagan) and a diagnostic-adjusted model.

1. The Bike Sharing daily dataset contains *731* rows and *16* columns.
2. The mean value of *cnt* is *4504.3488*.
3. Baseline model fit: $R^2 = 0.8381$ and *Adjusted* $R^2 = 0.8312$ (reported to 4 decimals).
4. In the baseline model, the predictor term with the smallest $P>|t|$ is *yr* (p-value shown as approximately 0.000 due to rounding).

5. Normality: *No*. The Jarque–Bera test rejects normality (p-value is effectively 0.0000), and the Q–Q plot shows systematic departures from the 45° line in the tails.
6. Multicollinearity: the largest VIF is *infinite* for *weekday_4* (and several related weekday dummy terms also show infinite VIF), indicating perfect multicollinearity in the baseline design matrix; report *VIF = inf*.
7. Autocorrelation: the Durbin–Watson statistic is *0.421* (rounded to 3 decimals). Because this value is far below 2.0, *positive autocorrelation is a concern* in this daily time-ordered dataset.
8. Linearity: the residuals-vs-fitted plot is *not purely random scatter*; it shows structure (a mild curve) and changing spread, suggesting the linearity/constant-variance assumptions are imperfect for the baseline specification.
9. Heteroscedasticity: the Breusch–Pagan test p-value is *0.0000* (rounded to 4 decimals). At $\alpha = 0.05$, *reject* homoscedasticity.
10. Diagnostic adjustment (example): remove *workingday* and *atemp* from the baseline predictor set to reduce multicollinearity (baseline VIFs include *inf* and very large values), and refit using *HC3 robust standard errors* to reduce sensitivity of inference to heteroscedasticity (Breusch–Pagan rejects constant variance).
11. Adjusted model fit (after dropping *workingday* and *atemp*, using HC3 SE): $R^2 = 0.8379$ and *Adjusted* $R^2 = 0.8313$. One diagnostic that improved is multicollinearity: the maximum VIF dropped from *inf* in the baseline model to *96.47* (rounded to 2 decimals) in the adjusted model.
12. In causal regression, you may accept a slightly worse fit if assumptions improve because the goal is *credible interpretation* of coefficients (and valid standard errors), not just maximizing explained variance. Diagnostics that reduce collinearity, stabilize residual behavior, or justify robust inference can make effect estimates more defensible even if R^2 changes little or declines slightly.

10.10Assignment

Complete the assignment below:

This assessment can be taken online.